OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 6

WRITE, DEBUG AND RUN PROGRAMS IN C

SUGGESTED TIME: 10 PERIODS

IN THIS CHAPTER

This chapter moves from algorithm design to real programming: students set up a C environment and learn to implement instructions, selection, iteration, strings, arrays, records and subroutines in the C language. Through progressive, exam-oriented practice, learners write, debug and run complete C programs that solve authentic problems, in line with the MINESEC Upper Sixth Computer Science syllabus.

- By the end of this chapter I can install, configure and use a C programming environment (compiler, editor/IDE) to create, compile and run a program.
- By the end of this chapter I can write a complete C program with the correct structure (preprocessor directives, main function, blocks) and implement basic instructions: declarations, assignments, input and output.
- By the end of this chapter I can implement selection constructs (if, if…else, else-if ladder, switch) to translate decision-making algorithms into C.
- By the end of this chapter I can implement iteration constructs (while, do…while, for) and choose the appropriate loop for a given problem.
- By the end of this chapter I can manipulate strings and characters using the C string library and character-handling functions.
- By the end of this chapter I can declare, fill, traverse and process one-dimensional and two-dimensional arrays with loops.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. Setting up the C environment and first programs
2. Basic instructions and selection
3. Iteration: loops in C
4. Strings, characters and one-dimensional arrays
5. Two-dimensional arrays, records and lists
6. Subroutines, debugging and integration
7. Integration activity
8. Methods and worked examples
9. Exercises
10. Solutions to the exercises
11. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can install, configure and use a C programming environment (compiler, editor/IDE) to create, compile and run a program.
- By the end of this chapter I can write a complete C program with the correct structure (preprocessor directives, main function, blocks) and implement basic instructions: declarations, assignments, input and output.
- By the end of this chapter I can implement selection constructs (if, if...else, else-if ladder, switch) to translate decision-making algorithms into C.
- By the end of this chapter I can implement iteration constructs (while, do...while, for) and choose the appropriate loop for a given problem.
- By the end of this chapter I can manipulate strings and characters using the C string library and character-handling functions.
- By the end of this chapter I can declare, fill, traverse and process one-dimensional and two-dimensional arrays with loops.
- By the end of this chapter I can define and use records (struct) and simple lists of records to model real-life entities.
- By the end of this chapter I can decompose a problem into subroutines (functions with parameters and return values) and implement them in C.
- By the end of this chapter I can debug a C program by reading compiler messages, tracing variables and correcting syntax and logic errors.
- By the end of this chapter I can integrate all these skills to design, code, test and document a complete C application.

PREREQUISITES

- Algorithmics from Lower Sixth: variables, assignment, sequence, selection and iteration in pseudocode
- Data types (integer, real, character, boolean) and simple expressions
- Basic computer literacy: files, folders, typing and saving text
- Notion of a compiler versus an interpreter (seen in the chapter on programming languages)
- Simple mathematics: arithmetic operators, comparison and logic (AND, OR, NOT)

1 SETTING UP THE C ENVIRONMENT AND FIRST PROGRAMS

At the Mokolo market in Yaoundé, a wholesaler still records daily sales of tomatoes, plantains and ndolé in a paper notebook; at the end of the month, adding up hundreds of lines by hand takes her two full evenings and errors creep in. Meanwhile, the mobile money agent next door prints his daily report in seconds because his till runs a program. What separates them is not luck: it is a few hundred lines of C code that store data, make decisions, repeat calculations and produce summaries. In this chapter you will learn to write, debug and run such programs yourself, turning the algorithms you designed in Lower Sixth into real, working software.

Our first problem is therefore very concrete: **how do we go from an algorithm on paper to a program that actually runs on a computer?** The answer begins with choosing a language and setting up the tools that translate our ideas into machine instructions.

1.1 Why C? The place of C among programming languages

A computer's processor understands only **machine code**: long sequences of binary instructions such as `10110000 01100001`. No human enjoys writing that. **Programming languages** were invented so that humans can write instructions in a readable form, called the **source code**, which is then translated into machine code. Languages fall into two great families according to how this translation happens:

- **Interpreted languages** (e.g. Python): a program called an interpreter reads and executes the source code line by line, every time the program runs.
- **Compiled languages** (e.g. C, C++): a program called a compiler translates the whole source code once into machine code, producing a stand-alone file that can then be run as many times as desired, very fast.

C is a **compiled language**. This gives it two decisive advantages. First, **speed**: because the translation is done once, in advance, C programs run at nearly the full speed of the processor — which is why operating systems (Windows, Linux), microcontrollers in phones, decoders and point-of-sale terminals are largely written in C. Second, **portability**: the same C source file can be compiled on a Windows laptop in Bamenda and on a Linux server in Douala; only the compiler changes, not the code.

DID YOU KNOW? **Dennis Ritchie and the birth of C**

The C language was created around 1972 by Dennis Ritchie at Bell Laboratories (USA), while he and Ken Thompson were building the UNIX operating system. C was designed to be small, close to the machine, yet portable. More than fifty years later it still ranks among the most widely used languages in the world, and languages such as C++, Java and C# borrowed its syntax. For GCE general knowledge, remember: C, Dennis Ritchie, Bell Labs, early 1970s.

1.2 Components of a C programming environment

To write and run C programs you need three kinds of tools working together:

- A **text editor** or an **IDE** (Integrated Development Environment) to type the source code. Popular free choices are **Code::Blocks** and **Dev-C++** (full IDEs that bundle an editor, a compiler and a debugger in one window) and **VS Code** (a powerful editor extended with plug-ins).
- The **compiler**, most commonly **GCC** (GNU Compiler Collection), which translates your source code into machine code.
- The **linker**, which combines your translated code with pre-written library code (for example the code behind `printf`) to produce the final program.

DEFINITION **Compiler, linker, executable file**

A **compiler** is a program that translates the entire source code of a program written in a high-level language (here C) into **object code** (machine code), reporting any syntax errors it finds. A **linker** is a program that joins one or more object files together with library code to produce an **executable file**: a stand-alone file (e.g. `program.exe` on Windows) that the operating system can load and run directly, without the compiler.

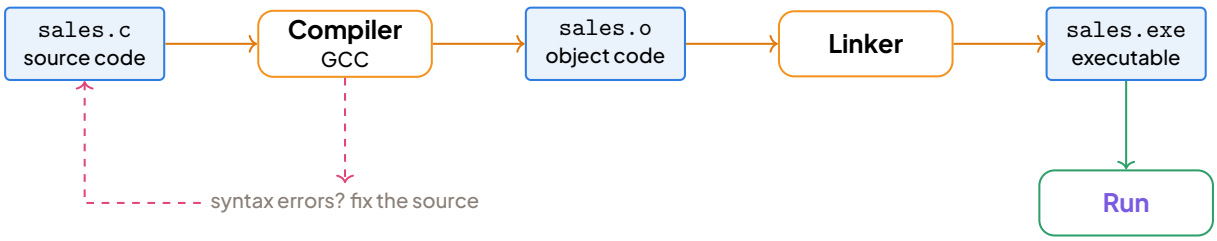
DEFINITION **IDE**

An **IDE** (Integrated Development Environment) is a single application that groups together all the tools a programmer needs: a text editor with syntax colouring, a compiler, a linker, a debugger and a button to run the program. Code::Blocks and Dev-C++ are the IDEs most often used in Cameroonian schools.

Installing the environment. On **Windows**, the simplest route is to download Code::Blocks with MinGW (MinGW is a Windows version of GCC); the installer configures everything automatically. On **Linux** (e.g. Ubuntu), open a terminal and type `sudo apt install gcc`; you can then edit files with any editor and compile from the terminal. In both cases, your program must be saved in a plain-text file whose name ends with the extension `.c`, for example `sales.c`.

1.3 The edit–compile–link–run cycle

Programming in C is a cycle. You **edit** the source file, you **compile** it into object code, the **linker** builds the executable, and you **run** it. If the compiler finds errors, you go back to editing. The diagram below summarises the cycle and the files involved.



The edit–compile–link–run cycle: the compiler turns `sales.c` into object code `sales.o`; the linker adds library code to build `sales.exe`; errors send you back to the editor.

METHOD **Five steps to create and run a C program**

1. Write the source code in the editor (or IDE).
2. Save the file with the `.c` extension, e.g. `hello.c`.
3. Compile it (in a terminal: `gcc hello.c -o hello`; in an IDE: the Build button).
4. Fix errors: read each compiler message (it gives the line number), correct the source, and recompile until there are no errors.
5. Run the executable (in a terminal: `./hello` or `hello.exe`; in an IDE: the Run button) and check that the output is correct.

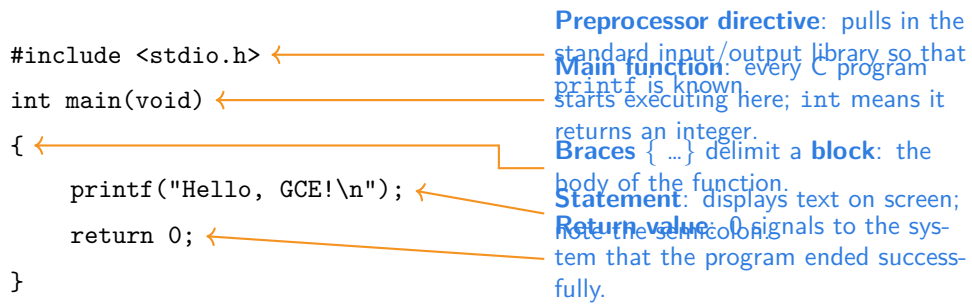
1.4 Anatomy of a first C program

Here is the smallest useful C program. Study it line by line; the figure labels each part.

EXEMPLE **The “Hello” program**

```
#include <stdio.h>


int main(void)
{
    printf("Hello, GCE!\n");
    return 0;
}
```



Annotated structure of a minimal C program.

Four rules govern the anatomy of every C program:

- A program is made of **functions**; `main` is the function where execution always begins.
- The body of a function is a **block** enclosed in braces `{ }`.
- Every **statement** inside a block ends with a **semicolon** ;.
- When `main` finishes, `return 0;` hands the value 0 back to the operating system, meaning "all went well".

WARNING **Semicolons and case-sensitivity**

Two traps catch every beginner. (1) Every statement ends with a semicolon. Forgetting it is the most common compile error; the compiler usually reports the error on the line after the missing semicolon. (2) C is case-sensitive: `main`, `Main` and `MAIN` are three different names. Writing `Main` instead of `main` means your program has no entry point and will not link. Keywords (`int`, `return`, `if` …) are always in lower case.

1.5 Displaying output with printf

`printf` ("print formatted") displays text and values on the screen. Its first argument is a **format string**: ordinary text mixed with **placeholders** (also called format specifiers) that are replaced by values, and **escape sequences** that control layout.

Placeholder	Displays a value of type …
`%d`	integer (`int`)
`%f`	real number (`float` or `double`)
`%c`	single character (`char`)
`%s`	string of characters

The most useful escape sequences are `\n` (new line) and `\t` (horizontal tab, for aligning columns).

EXEMPLE **Placeholders in action**

```
int bags = 12;

float price = 2500.5;

printf("Bags: %d\n", bags);        // displays: Bags: 12

printf("Price: %.2f FCFA\n", price); // displays: Price: 2500.50 FCFA
```

Placeholders are replaced in order by the values listed after the format string. `%.2f` rounds a real number to two decimal places.

1.6 Reading input with scanf

`scanf` reads values typed at the keyboard into variables. It uses the same format specifiers as `printf`, but with one crucial difference: you must give it the **address** of each variable, using the **address-of operator** `&`.

EXEMPLE **Reading a quantity**

```
int quantity;

printf("Enter quantity sold: ");

scanf("%d", &quantity);  // note the & !

printf("You sold %d units.\n", quantity);
```

Why the `&`? `scanf` must write into your variable; to do that it needs to know where the variable lives in memory, i.e. its address. `quantity` is the value; `&quantity` is the address.

WARNING **scanf and the missing &**

Writing `scanf("%d", quantity);` without `&` is a classic error. The compiler may only warn, but at run time the program typically crashes (or corrupts memory) because `scanf` writes to a nonsense address. Other input pitfalls: using the wrong specifier (`%d` for a `float` — use `%f`), and expecting `scanf("%s", …)` to read spaces (it stops at the first space).

1.7 Comments, indentation and good style

Programs are read by humans far more often than by computers. Two habits make your code readable — and earn marks at the GCE:

- **Comments** explain why the code does something. `//` starts a comment to the end of the

line; `/* … */` delimits a comment spanning several lines. Comments are ignored by the compiler.

- **Indentation**: lines inside a block are shifted right (one tab or 4 spaces), so the structure is visible at a glance.

EXEMPLE **Well-presented code**

```
/* Program:  daily sales total    */
   Author:  A. Ndi, Upper Sixth */
#include <stdio.h>
int main(void)
{
    int qty;              // quantity sold today
    scanf("%d", &qty);    // read it from the keyboard
    printf("Total:  %d\n", qty);
    return 0;
}
```

1.8 First debugging session: reading compiler errors

A **bug** is an error in a program; **debugging** is the process of finding and fixing bugs. The compiler is your first ally: each message gives the **file name**, the **line number** and a description.

WORKED EXAMPLE 1 **Debugging a broken program**

The following program is submitted to GCC:

```
#include <stdio.h>
int main(void)
{
    int total
    total = price * 2;
    printf("%d\n", total);
    return 0;
```

```
}
```

The compiler answers:

```
sales.c: In function 'main':
sales.c:5:5: error: expected ';' before 'total'
sales.c:5:12: error: 'price' undeclared (first use in this function)
```

Correct the program.

SOLUTION

Solution. Read the messages in order:

1. `line 5: expected ';' before 'total'` — the declaration on line 4, `int total`, is missing its semicolon. Note how the compiler reports the error on the next line.
2. `'price' undeclared` — the variable `price` is used but was never declared. Every variable must be declared before use.

Corrected program:

```
#include <stdio.h>

int main(void)

{

    int total;          // semicolon added

    int price = 1500;   // price declared and initialised

    total = price * 2;

    printf("%d\n", total);  // displays: 3000

    return 0;

}
```

The program now compiles cleanly and displays `3000`.

KEY POINTS **Key points of this section**

- C is a compiled, fast, portable language, created by Dennis Ritchie; it dominates systems and embedded programming.
- The toolchain is: editor/IDE → compiler (source `.c` → object `.o`) → linker (→ executable) → run.
- Every C program has a `main` function; statements end with `;`; blocks use `{ }`; C is case-sensitive.

- `printf` displays using placeholders (`%d`, `%f`, `%c`, `%s`) and escapes (`\n`, `\t`); `scanf` reads into variables and requires `&`.
- Compiler error messages give a line number and a cause — read them from the top and fix one at a time.

EXERCISE 1 **GCE-style practice**

1. Define: compiler; linker; executable file.
2. A student writes `Int Main(void)` at the top of a program. Give two distinct reasons why this fails.
3. What is wrong with each of these lines? (a) `printf("Sum = %d", sum)` (b) `scanf("%d", age);` (c) `int 2total;`
4. Write a complete C program that asks the user for the number of crates of plantain sold and the price per crate, then displays the total amount in FCFA.

SOLUTION **Exercise 1**

1. Compiler: translates the whole C source code into object (machine) code and reports syntax errors. Linker: joins object files with library code to build the executable. Executable file: stand-alone machine-code file the operating system can run directly.
2. (i) `Int` with a capital I is not the keyword `int` (C is case-sensitive); (ii) `Main` is not `main`, so the program has no entry point and linking fails.
3. (a) Missing semicolon at the end. (b) Missing `&`: it should be `scanf("%d", &age);`. (c) An identifier may not start with a digit.
4. One correct solution:

```
#include <stdio.h>
int main(void)
{
    int crates;
    float price;
    printf("Crates sold: ");
    scanf("%d", &crates);
```

```
    printf("Price per crate: ");
    scanf("%f", &price);
    printf("Total: %.2f FCFA\n", crates * price);
    return 0;
}
```

WARNING **GCE tip: labelling a program**

In theory papers you may be given a short C program and asked to label its parts: preprocessor directive (`#include`), header of `main`, opening/closing braces, statement, comment, `return` statement. Practise naming these on the "Hello" program above — it is an easy mark.

2 BASIC INSTRUCTIONS AND SELECTION

In the previous section you installed a C compiler, wrote your first program with `printf`, and learned the compile–run cycle. That program could only display fixed text: it had no memory of its own and could take no decisions. Real programs must store values, compute with them, and choose between alternative actions. In this section we learn how C handles these two fundamental abilities: **basic instructions** (declarations, assignments, arithmetic) and **selection** (decision-making with `if`, `else if` and `switch`).

2.1 Variables, data types and constants

Observation. Imagine a program that computes the school fees balance of a student in a Yaoundé lycée. The program must remember the total fees, the amount already paid, and the student's name initial. Each of these pieces of information needs a named place in memory.

DEFINITION **Variable, data type and constant**

A **variable** is a named memory location whose value can change during program execution. Every variable in C has a **data type**, which fixes the kind of value it can hold and how much memory it occupies. A **constant** is a named value that cannot change during execution.

The fundamental C types you must know at Advanced Level are:

Type	Stores	Example values
`int`	Whole numbers (typically 4 bytes)	`7, -25, 2024`
`float`	Real numbers, single precision (4 bytes)	`3.5, -0.25`
`double`	Real numbers, double precision (8 bytes)	`3.141592653589`
`char`	A single character (1 byte)	`'A', '7', '%'`

Declaring and initialising. A variable must be declared before use, and it is good practice to give it an initial value:

- `int age = 17;`
- `float fees = 50000.0;`
- `double average = 12.75;`
- `char grade = 'B';`

Naming rules. An **identifier** (variable name) may contain letters, digits and the underscore `_`, but must **not begin with a digit**. C is case-sensitive: `mark` and `Mark` are two different variables. Reserved words such as `int`, `if` and `return` cannot be used as names. Choose meaningful identifiers: `totalFees` is far clearer than `x`.

Constants. C offers two ways to define constants:

- `#define PI 3.14159` (preprocessor directive, no semicolon, placed before `main`)
- `const int MAX_MARK = 100;` (typed constant, protected by the compiler)

DID YOU KNOW? **C in Cameroon**

C is the backbone of many systems you use daily: the firmware in mobile money terminals (MTN MoMo, Orange Money), the embedded software in point-of-sale devices across Douala markets, and the core of the Linux kernel running on servers that host Cameroon's educational platforms. Mastering C opens doors to systems programming, embedded development, and performance-critical applications.

WARNING **Uninitialised variables**

A variable declared without an initial value contains whatever bits happened to be in memory — a garbage value. Writing `int total; printf("%d", total);` may print any number. Always initialise your variables.

2.2 Assignment and arithmetic operators

The **assignment** operator = copies the value on the right into the variable on the left:

- `fees = 75000.0;` // fees now holds 75000.0

- `balance = fees - paid;` // compute, then store

C provides five arithmetic operators: + (addition), - (subtraction), * (multiplication), / (division) and % (**modulo**: the remainder of an integer division, e.g. `7 % 2` is `1`).

PROPERTY **Integer division**

In C, the operator / applied to two integers performs integer division: the fractional part is discarded. Thus `7 / 2` gives `3`, not `3.5`. To obtain real division, at least one operand must be a real number: `7.0 / 2` gives `3.5`.

Operator precedence. *, / and % are evaluated before + and -; operators of equal precedence are evaluated left to right; parentheses override everything. Hence `2 + 3 * 4` is `14`, while `(2 + 3) * 4` is `20`.

EXEMPLE **Average of three marks**

To compute the average of three marks `m1`, `m2`, `m3` (all `int`):

- `float average = (m1 + m2 + m3) / 3.0;`

We divide by `3.0` (a `double` literal) to force real division. Writing `/ 3` would truncate the result.

Compound assignment and increment. C provides shortcuts for very common updates:

Shortcut	Equivalent long form
`x += 5;`	`x = x + 5;`
`x -= 2;`	`x = x - 2;`
`x *= 3;`	`x = x * 3;`
`x++;`	`x = x + 1;`
`x--;`	`x = x - 1;`

For example, a counter of students present is updated with `count++;` each time a student enters.

2.3 Type conversion and casting

When an expression mixes types, C performs **implicit conversion**: the "smaller" type is automatically promoted (e.g. in `7 / 2.0`, the `7` becomes `7.0` before dividing). You can also force a conversion with an **explicit cast**:

- `int x = 7;`
- `float half = (float)x / 2;` // x is converted to 7.0, result is 3.5

WARNING **Truncation danger**

Casting a real number to `int` truncates (cuts off) the fractional part; it does not round: `(int)3.9` gives `3`. Assigning a `float` into an `int` variable truncates silently in the same way. Also note: `(float)(7 / 2)` gives `3.0`, because the integer division happens before the cast — write `(float)7 / 2` instead.

2.4 Relational and logical operators

Selection needs conditions. A condition is an expression built from **relational operators**:

Operator	Meaning	Example (true when $x = 5$)
==	equal to	`x == 5`
!=	not equal to	`x != 3`
<	less than	`x < 10`
<=	less than or equal	`x <= 5`
>	greater than	`x > 2`
>=	greater than or equal	`x >= 5`

DEFINITION **Truth values in C**

C has no separate Boolean type. A condition that is false evaluates to `0`; a condition that is true evaluates to a non-zero value (by convention `1`). Consequently, any non-zero value is treated as true inside a condition.

Conditions are combined with **logical operators**:

- `&&` (AND): true only if both sides are true. `(mark >= 0) && (mark <= 20)`
- `||` (OR): true if at least one side is true. `(day == 6) || (day == 7)`
- `!` (NOT): reverses the truth value. `!(x > 5)` means $x \leq 5$.

WARNING **= versus ==**

Assignment uses a single `=`; comparison uses a double `==`. Writing `if (x = 5)` does not test whether x equals 5: it assigns 5 to `x`, and since 5 is non-zero the condition is always true. This is one of the most classic logic errors in C — the compiler often accepts it silently, and the program then behaves wrongly.

2.5 Simple selection: if and if...else

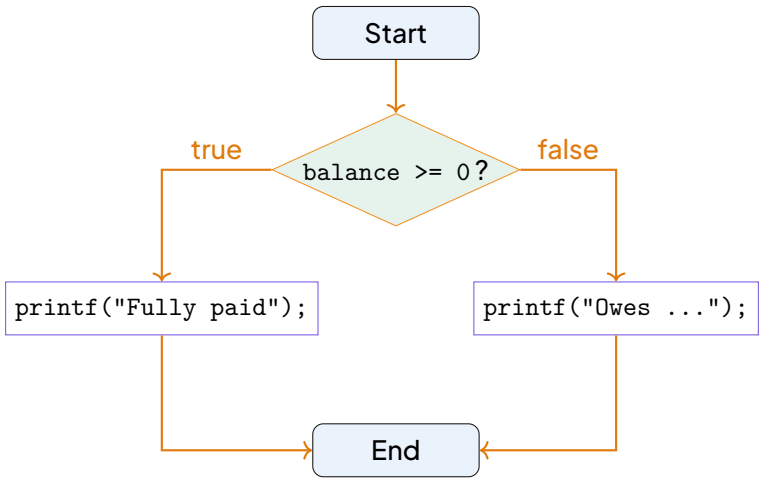
In pseudocode you learned to write decisions such as "IF balance ≥ 0 THEN display 'Paid' ELSE display 'Owes'". In C this becomes:

```
if (balance >= 0)
  printf("Fully paid\n");
else
  printf("Owes %.0f FCFA\n", -balance);
```

The `else` part is optional: a simple `if` executes its block only when the condition is true. When a branch contains more than one instruction, the instructions must be grouped inside braces `{ }`.

METHOD **Translating a pseudocode selection into C**

1. Identify the condition and write it with relational/logical operators (remember `==`, never `=`).
2. Write `if (condition) {` then translate the THEN instructions, then close with `}`.
3. If there is an ELSE part, add `else {`, translate the ELSE instructions, close with `}`.
4. Check every branch: does each one do exactly what the pseudocode says?
5. Test with values on each side of the boundary (e.g. just below, equal to, and just above the threshold).



Flowchart of an `if...else` decision: the diamond tests the condition, and exactly one of the two branches is executed before the flow rejoins.

2.6 Multiple selection: else-if ladder and switch

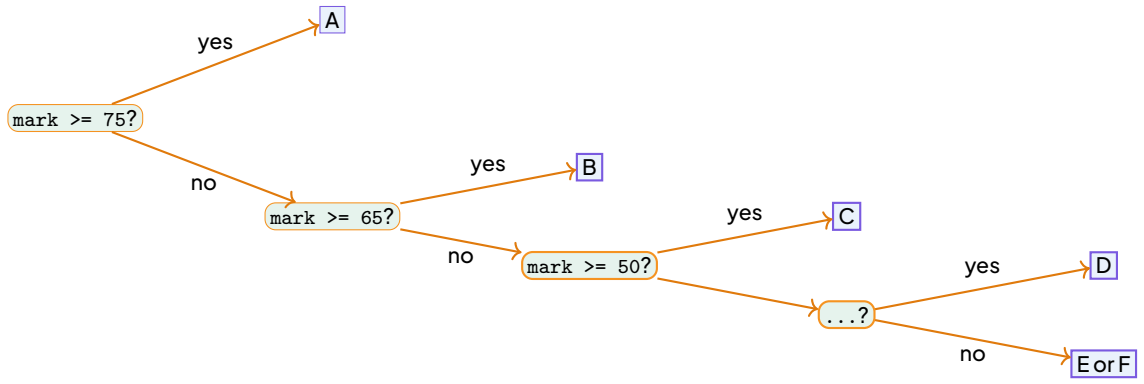
When there are more than two cases, chain tests with an **else-if ladder**. The conditions are tested from top to bottom; as soon as one is true, its block runs and the rest are skipped.

EXEMPLE **Grading a GCE mark**

A mark out of 100 is graded: A (75–100), B (65–74), C (50–64), D (40–49), E (30–39), F (below 30). (Note: this scale is illustrative; the official Cameroon GCE A-Level grading uses slightly different boundaries.)

- `if (mark >= 75) grade = 'A';`
- `else if (mark >= 65) grade = 'B';`
- `else if (mark >= 50) grade = 'C';`
- `else if (mark >= 40) grade = 'D';`
- `else if (mark >= 30) grade = 'E';`
- `else grade = 'F';`

Notice the economy of the ladder: because earlier tests failed, `mark >= 65` alone is enough for grade B — we already know `mark < 75`.



Decision tree for the GCE grading ladder: each "yes" branch assigns a grade, each "no" branch moves to the next test.

The switch statement. When a single variable is compared against several discrete values (integers or characters), `switch` is clearer than a ladder:

- `switch (choice) {`
- `  case 1: printf("Create\n"); break;`
- `  case 2: printf("Edit\n"); break;`

```
    case 3: printf("Quit\n"); break;
    default: printf("Invalid choice\n");
}
```

The value of `choice` is compared with each `case`; the matching block runs, and `break` exits the `switch`. The optional `default` catches all other values. A `switch` cannot test ranges such as `mark >= 75` — for ranges, use the else-if ladder.

WARNING **Forgetting break causes fall-through**

If you omit `break`, execution falls through into the next case. With `choice = 1` and no `break` after `case 1`, the program prints "Create", then "Edit", then "Quit", then "Invalid choice"! Always end each case with `break` unless fall-through is deliberately intended.

EXPERIMENT **Guided lab: menu-driven calculator**

Write a complete C program that displays a menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Quit

Read the user's choice (1–5) with `scanf`, then read two `float` operands. Use a `switch` to perform the selected operation and print the result. For division, check that the second operand is not zero. Compile with `gcc -Wall -o calc calc.c` and run with `./calc`. Test each option, including invalid choices and division by zero.

2.7 Nested selection and compound conditions

Selections can be **nested**: an `if` inside another `if`. For example, to check that a mark is valid before grading it:

```
if (mark >= 0 && mark <= 100) {
    if (mark >= 50) printf("Pass\n");
    else printf("Fail\n");
} else {
    printf("Invalid mark\n");
```

```
}
```

The outer condition uses the **compound condition** `mark >= 0 && mark <= 100`: both tests must hold. Note that C has no chained comparison — `0 <= mark <= 100` does not work as in mathematics (it is parsed as `(0 <= mark) <= 100`, which is always true because the result of `0 <= mark` is 0 or 1).

2.8 Debugging selection

A decision that compiles may still be wrong. Two habits save you:

1. **Trace with printed intermediate values.** Insert temporary lines such as `printf("DEBUG mark=%d\n", mark);` before the selection, run the program, and check that the values are what you expect. Remove the debug lines once the fault is found.
2. **Test boundary cases.** For the grading ladder, test 74, 75 (A boundary), 64, 65 (B boundary), 0, 100, and invalid values such as -5 and 101. Most selection bugs hide exactly at the boundaries, often because > was written instead of >=.

WORKED EXAMPLE 2 **Worked exercise: largest of three numbers**

Write a C fragment that reads three integers `a`, `b`, `c` and prints the largest.

SOLUTION

Solution. We use an else-if ladder with compound conditions:

```
if (a >= b && a >= c)
  printf("Largest = %d\n", a);
else if (b >= a && b >= c)
  printf("Largest = %d\n", b);
else
  printf("Largest = %d\n", c);
```

Trace with `a = 7, b = 12, c = 9`: the first condition `7 >= 12 && ...` is false; the second `12 >= 7 && 12 >= 9` is true, so the program prints `Largest = 12`, which is correct. Boundary test `a = 5, b = 5, c = 2` prints `Largest = 5`: ties are handled gracefully because we used >=.

EXERCISE 2 **Practice**

1. A shop in Bamenda gives a 5% discount if the purchase exceeds 50 000 FCFA. Write a C fragment that reads a `float amount`, applies the discount when due, and prints the

final price.

2. Convert the GCE grading ladder into a program that first rejects marks outside [0, 100], then prints the grade. Test it with 75, 30, 29 and 150.
3. Rewrite the menu `switch` above so that choosing 1, 2 or 3 prints the number of the option squared; then deliberately remove one `break`, predict the output for `choice = 2`, and verify by running the program.

SOLUTION **Exercise 1**

```
float amount, final;
scanf("%f", &amount);
if (amount > 50000)
  final = amount * 0.95;
else
  final = amount;
printf("Final price: %.0f FCFA\n", final);
```

The condition uses > (strictly exceeds); `amount * 0.95` applies the 5% discount in real arithmetic.

KEY POINTS **Key points**

- Declare every variable with a type (`int`, `float`, `double`, `char`) and initialise it; use `const` or `#define` for constants.
- `7 / 2` is `3` (integer division); force real division with `7.0 / 2` or a cast `(float)x / 2`. Casts truncate, they never round.
- Comparison is `==`; assignment is `=`. In C, 0 means false and any non-zero value means true.
- Use `if...else` for two cases, an else-if ladder for ranges, and `switch` for discrete values — never forgetting `break`.
- Debug decisions by printing intermediate values and testing boundary cases.

With basic instructions and selection mastered, your programs can now store data, compute and take decisions. The next step is to make the computer repeat actions — that is the subject

of iteration, which we study in the following section.

3 ITERATION: LOOPS IN C

In the previous section we saw how `if` and `switch` let a program choose between alternative paths. Many real problems, however, require repetition: reading all the marks of a class, printing a multiplication table, or searching a list until a value is found. Writing the same statements over and over is tedious, error-prone and impossible when the number of repetitions is not known in advance. This section introduces the three loop constructs of C — `while`, `do...while` and `for` — and shows how to use them correctly, trace them by hand and avoid the classic pitfalls that appear every year in the GCE Advanced Level examination.

3.1 The need for repetition: from pseudocode to C loops

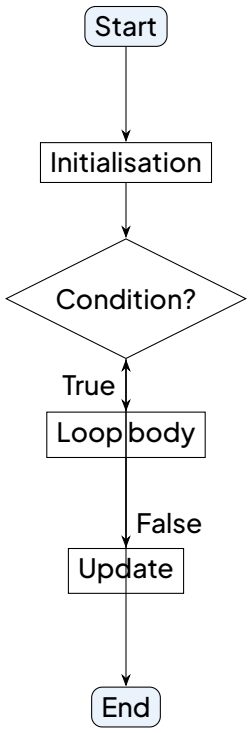
Every loop, whether in pseudocode or in C, has three logical parts:

- **Initialisation** — setting up variables before the first iteration (e.g. `sum = 0; i = 1;`).
- **Condition** — a Boolean expression that is tested to decide whether the loop continues.
- **Update** — modifying one or more variables so that the condition eventually becomes false (e.g. `i++` or `sum += mark`).

In the GCE pseudocode you know three loop structures:

Pseudocode	C equivalent
`WHILE condition DO ... END WHILE`	`while (condition) { ... }`
`REPEAT ... UNTIL condition`	`do { ... } while (condition);`
`FOR var FROM start TO end STEP step DO ... END FOR`	`for (init; condition; update) { ... }`

The C `for` loop is more flexible than the pseudocode `FOR` because the three parts are arbitrary expressions, not just a simple counter.



Generic loop flow: initialisation → test → body → update → test…

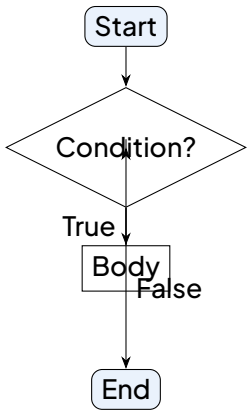
3.2 The while loop

DEFINITION **while loop**

The `while` loop repeats a block of statements as long as its condition evaluates to true (non-zero). The condition is tested before each iteration, so the body may never execute if the condition is initially false.

Syntax:

```
while (condition) {
    // body statements
}
```



Flowchart of a `while` loop

EXEMPLE **Sentinel-controlled input: reading marks until -1**

A teacher wants to enter the marks of an unknown number of students. The entry stops when the user types `-1`. We need the sum and the count to compute the average later.

```
int mark, sum = 0, count = 0;
printf("Enter marks (-1 to stop): ");
scanf("%d", &mark);   // priming read
while (mark != -1) {
    sum += mark;
    count++;
    scanf("%d", &mark);   // next read
}
if (count > 0)
    printf("Average = %.2f\n", (float)sum / count);
else
    printf("No marks entered.\n");
```

WORKED EXAMPLE 3 **Trace of the sentinel loop**

Suppose the user enters: `12 15 10 -1`. Trace the variables `mark`, `sum`, `count` after each iteration.

Step	mark	sum	count
Before loop (priming read)	12	0	0
Iteration 1	12	12	1
Read next	15	12	1
Iteration 2	15	27	2
Read next	10	27	2
Iteration 3	10	37	3
Read next	-1	37	3
Condition false, exit	-1	37	3

Result: Average = 37/3 = 12.33.

WARNING **Empty loop body**

A semicolon immediately after the condition creates an empty body:

```
while (i < 10);   // infinite loop if i < 10 initially!
```

The compiler sees a null statement as the body; `i` is never updated. Always use braces even for a single statement.

3.3 The do...while loop

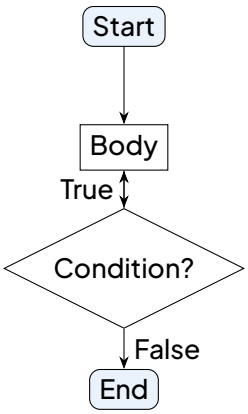
DEFINITION **do...while loop**

The `do...while` loop executes its body at least once because the condition is tested after the body. It is the direct translation of the pseudocode `REPEAT ... UNTIL`.

Syntax:

```
do {
    // body statements
} while (condition);
```

Note the mandatory semicolon after the condition.



Flowchart of a `do...while` loop

EXEMPLE **Menu-driven program**

A classic use of `do...while` is a menu that must appear at least once:

```
int choice;
do {
    printf("\n1. Add\n2. Subtract\n3. Quit\nChoice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1: /* add */ break;
        case 2: /* subtract */ break;
        case 3: printf("Goodbye!\n"); break;
        default: printf("Invalid option.\n");
    }
} while (choice != 3);
```

WARNING **do...while vs while**

`do...while` always runs the body once, even if the condition is false initially. `while` may run zero times. Choose `do...while` when the first execution is mandatory (menus, input validation).

3.4 The for loop

DEFINITION **for loop**

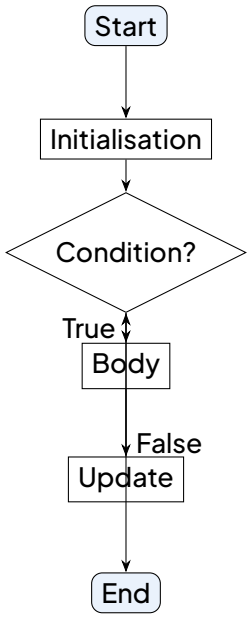
The `for` loop gathers initialisation, condition and update in a single line. It is ideal when the number of iterations is known or can be computed before the loop starts.

Syntax:

```
for (initialisation; condition; update) {
    // body
}
```

Execution order:

1. `initialisation` runs **once** before the first test.
2. `condition` is evaluated; if false, the loop ends.
3. `body` executes.
4. `update` executes.
5. Go back to step 2.



Flowchart of a `for` loop

EXEMPLE **Variations of the for loop**

- Counting up: `for (int i = 1; i <= 10; i++)`

- Counting down: `for (int i = 10; i >= 1; i--)`
- Step 2: `for (int i = 0; i <= 100; i += 2)`
- Multiple variables: `for (int i=0, j=10; i<j; i++, j--)`
- Empty parts: `for (; condition; )` is equivalent to `while (condition)`.

WARNING **Semicolon after for**

`for (int i=0; i<10; i++);` // empty body! The loop runs 10 times doing nothing, then continues after the semicolon. This is a very common GCE trap.

3.5 Choosing the right loop

KEY POINTS **Loop selection checklist**

- Known number of iterations (counting, fixed-size arrays) → `for`.
- Condition-controlled, may run zero times (sentinel input, file reading) → `while`.
- Condition-controlled, must run at least once (menus, input validation) → `do...while`.

DID YOU KNOW? **GCE tip**

Examiners often ask: "Convert the following while loop into an equivalent for loop" or "Trace the execution of this loop and give the final values of the variables." Practise both conversions and hand-tracing with a trace table.

3.6 Accumulators and counters inside loops

DEFINITION **Counter and accumulator**

A counter is a variable incremented (or decremented) by a fixed amount each iteration to track how many times the loop has run. An accumulator gathers a running total (sum, product, concatenation, etc.) by combining each new value with the previous total.

DEFINITION **Loop invariant**

A loop invariant is a condition that is true before the loop starts, before each iteration and after the loop terminates. It helps prove correctness. Example: in a sum loop, `sum == sum of all marks processed so far` is the invariant.

EXEMPLE **Sum, average, maximum and minimum**

Read marks until -1; compute sum, average, max and min.

```
int mark, sum = 0, count = 0, max = -1, min = 101;
printf("Enter marks (-1 to stop): ");
scanf("%d", &mark);
while (mark != -1) {
    sum += mark;
    count++;
    if (mark > max) max = mark;
    if (mark < min) min = mark;
    scanf("%d", &mark);
}
if (count > 0) {
    printf("Sum = %d, Average = %.2f\n", sum, (float)sum/count);
    printf("Max = %d, Min = %d\n", max, min);
}
```

Step	mark	sum	count	max	min
Start	-	0	0	-1	101
Read 12	12	0	0	-1	101
Iter 1	12	12	1	12	12
Read 15	15	12	1	12	12
Iter 2	15	27	2	15	12
Read -1	-1	27	2	15	12

Trace table for sum, average, max, min loop (input: 12, 15, -1)

3.7 Nested loops

A **nested loop** places one loop completely inside the body of another. The inner loop runs to completion for each iteration of the outer loop.

EXEMPLE **Multiplication table 1..5**

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= 5; j++) {
        printf("%4d", i * j);
    }
    printf("\n");
}
```

Output:

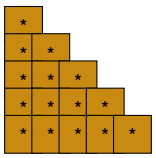
```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

EXEMPLE **Star triangle pattern**

```
for (int row = 1; row <= 5; row++) {
    for (int star = 1; star <= row; star++) {
        printf("*");
    }
    printf("\n");
}
```

Output:

```
*
**
***
****
*****
```



Visual representation of the star triangle produced by the nested loop

METHOD **Tracing a nested loop by hand**

1. Write down the initial values of all loop variables.
2. Execute the outer loop initialisation.
3. Test outer condition; if false, stop.
4. Enter inner loop: initialise inner variable, test inner condition.
5. While inner condition true: execute inner body, update inner variable, test again.
6. When inner loop finishes, execute any remaining outer-body statements (e.g. `printf("\n");`).
7. Update outer variable, go back to step 3.

Use a trace table with columns for `row`, `star`, and output.

3.8 Loop control: break and continue

DEFINITION **break and continue**

- `break;` — immediately exits the innermost loop (or `switch`).
- `continue;` — skips the rest of the current iteration and jumps to the update part (in `for`) or the condition test (in `while`/`do...while`).

EXEMPLE **Using break to exit early**

Search for the first negative number in an array:

```
int a[] = {5, 12, -3, 7, 9};
int i;
for (i = 0; i < 5; i++) {
    if (a[i] < 0) {
        printf("First negative at index %d\n", i);
        break;
    }
}
if (i == 5) printf("No negative found.\n");
```

EXEMPLE **Using continue to skip an iteration**

Print only odd numbers from 1 to 10:

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   // skip even numbers
    printf("%d ", i);
}
```

Output: `1 3 5 7 9`

WARNING **Infinite loops**

An infinite loop occurs when the condition never becomes false. Common causes:

- Forgetting the update: `while (i < 10) { printf("%d", i); }` // missing i++
- Wrong update direction: `for (int i=0; i<10; i--)`
- Condition always true: `while (1) { ... }` without a `break`.

Always verify that the update moves the condition toward false.

3.9 Debugging loops: off-by-one errors and tracing tables

KEY POINTS **Common loop bugs**

- Off-by-one: Using <= instead of < (or vice-versa) when indexing arrays of size `N` (valid

indices 0..N-1).

- Wrong initialisation: `sum = 1` instead of `0`; `max = 0` when all values are negative.
- Update in wrong place: In a `while` loop, placing the update before the body can skip the first value or process an extra value.
- Semicolon after loop header: Creates empty body (see warnings above).

METHOD **Systematic loop debugging**

1. Write a trace table with columns for every variable that changes.
2. Execute the loop by hand for 2–3 iterations, then for the boundary (last) iteration.
3. Check the loop invariant at the start of each iteration.
4. Verify the termination condition: will it eventually become false?
5. Test with edge cases: empty input (zero iterations), one iteration, maximum size.

WORKED EXAMPLE 4 **Debugging an off-by-one error**

A student writes the following to compute the sum of the first 10 positive integers:

```
int sum = 0;
for (int i = 1; i < 10; i++) {   // BUG: should be i <= 10
    sum += i;
}
printf("Sum = %d\n", sum);   // prints 45 instead of 55
```

Trace table:

i	sum (before)	sum (after)
1	0	1
2	1	3
3	3	6
4	6	10
5	10	15
6	15	21
7	21	28
8	28	36
9	36	45
10	loop stops	45

Fix: Change condition to `i <= 10` or `i < 11`.

EXERCISE 3 **Practice: loop tracing and conversion**

1. Convert the following `while` loop into an equivalent `for` loop:

```
int i = 1, prod = 1;
  while (i <= 5) {
       prod *= i;
          i++;
         }
```

2. Trace the nested loop below and draw the exact output pattern:

```
for (int r = 5; r >= 1; r--) {
  for (int c = 1; c <= r; c++) {
           printf("%d", c);
                }
          printf("\n");
              }
```

3. Identify the error in this code meant to read positive numbers until a negative is entered, then print the count:

```
int x, count = 0;
 scanf("%d", &x);
while (x >= 0); {
       count++;
```

```
        scanf("%d", &x);
    }
printf("Count = %d\n", count);
```

SOLUTION **Exercise 1**

1. `for (int i = 1, prod = 1; i <= 5; i++) { prod *= i; }`

2. Output:

```
12345
1234
123
12
1
```

3. The semicolon after `while (x >= 0);` creates an empty loop body. If the first input is non-negative, the program hangs in an infinite loop. Remove the semicolon.

KEY POINTS **Key points for the exam**

- `while` — test first, may run zero times.
- `do...while` — test last, runs at least once.
- `for` — initialise, test, update in one line; best for known counts.
- `break` exits the loop; `continue` skips to next iteration.
- Always initialise accumulators (`sum=0`, `prod=1`) and counters (`count=0`) before the loop.
- Trace tables are your best friend: use them in the exam to avoid off-by-one errors.
- GCE favourite: convert between loop types, trace nested loops, spot the empty-body semicolon.

4 STRINGS, CHARACTERS AND ONE-DIMENSIONAL ARRAYS

In the previous section we mastered loops: `while`, `do...while` and `for`. Loops become truly powerful when they process collections of data. Imagine the form master of an Upper Sixth class in Bamenda who must store the names of 45 students, count the vowels in each name, or find the highest mark in the class. Doing this with 45 separate variables would be absurd. C gives us two natural tools: the **string** (a collection of characters) and the **one-dimensional array** (a collection of values of the same type). Both are processed character by character or cell by cell — with the loops we have just learned.

4.1 The `char` type and ASCII codes

A variable of type `char` stores a single character: a letter, a digit or a symbol. Internally, the computer stores only numbers, so every character is represented by an integer code. The standard coding table is the **ASCII** table (American Standard Code for Information Interchange).

DEFINITION **The `char` type and ASCII**

A `char` variable occupies one byte and stores the integer code of a character. Character constants are written between single quotes: `'A'`, `'7'`, `'#'`. Useful ASCII codes: `'A'` is 65, `'Z'` is 90, `'a'` is 97, `'z'` is 122, `'0'` is 48 and `'9'` is 57.

Because a `char` is a small integer, we can do arithmetic on it: `'A' + 1` gives `'B'`, and `'7' - '0'` gives the integer 7. This is exactly how we convert a digit character into its numeric value.

The header `<ctype.h>` provides ready-made character functions:

Function	Meaning
`isdigit(c)`	non-zero (true) if `c` is a digit `'0'` to `'9'`
`isalpha(c)`	non-zero if `c` is a letter
`toupper(c)`	uppercase version of `c` (unchanged if not a letter)
`tolower(c)`	lowercase version of `c`

EXEMPLE **Normalising a character**

To accept both `'y'` and `'Y'` as an answer:

- `#include <ctype.h>`
- `char reply; scanf(" %c", &reply);`
- `if (toupper(reply) == 'Y') { printf("Yes chosen"); }`

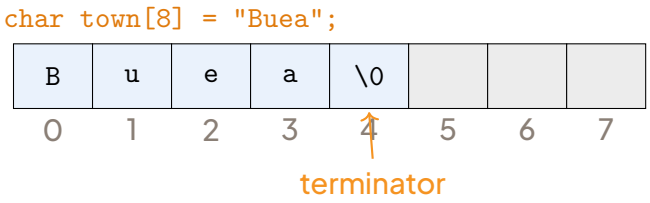
4.2 Strings in C: arrays of `char` ending with `'\0'`

DEFINITION **String and null terminator**

In C, a **string** is an array of `char` whose last meaningful character is followed by the null character `'\0'` (ASCII code 0). This **null terminator** marks the end of the string; every string function relies on it to know where the text stops.

Declaration and initialisation:

- `char name[20];` (space for 19 characters + the terminator)
- `char town[] = "Buea";` (compiler counts: 5 cells, `B u e a \0`)
- `char code[8] = "GCE";` (remaining cells are filled with `'\0'`)



Memory layout of a string: each character occupies one cell; `'\0'` ends the string; unused cells may contain garbage.

4.3 Reading and printing strings

To print a string, use `printf` with `%s`: `printf("Town: %s", town);`

For input, `scanf("%s", name);` works but has two serious limits:

1. it stops at the first space, so `"Ngo Bassa"` is read as just `"Ngo"`;
2. it does not check the array size: typing more characters than the array can hold causes a buffer overflow, a classic security bug.

The safe alternative is `fgets`, which reads a whole line, spaces included, and never exceeds a given size:

- `char name[30];`
- `fgets(name, 30, stdin);`

WARNING `fgets` **keeps the newline**

`fgets` stores the `'\n'` produced by the Enter key just before the `'\0'`. Remove it when nec-

essary: `name[strlen(name)-1] = '\0';`

4.4 The <string.h> library

Function	Effect
`strlen(s)`	number of characters before `'\0'`
`strcpy(dest, src)`	copies `src` into `dest`
`strcat(dest, src)`	appends `src` at the end of `dest`
`strcmp(a, b)`	0 if equal, negative if `a < b`, positive if `a > b`

WARNING **Never use = or == on strings**

Writing `name = "Ali";` is a compile error, and `if (a == b)` compares addresses, not contents — it is almost always false even for identical texts. To copy, use `strcpy(name, "Ali");` and to compare, use `if (strcmp(a, b) == 0)`.

EXEMPLE **Using `strcpy`, `strcat` and `strcmp`**

```
char s1[20] = "Hello";
char s2[20] = "World";
char s3[40];
strcpy(s3, s1);   // s3 = "Hello"
strcat(s3, " ");   // s3 = "Hello "
strcat(s3, s2);   // s3 = "Hello World"
if (strcmp(s1, s2) < 0) printf("s1 comes before s2");
```

4.5 Processing a string character by character

Since a string is an array, a `for` loop from index 0 to `strlen(s)-1` visits every character. Three classic applications:

Counting vowels.

```
int i, count = 0;
for (i = 0; i < strlen(s); i++) {
  char c = tolower(s[i]);
```

```
  if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
}
```

Reversing a name (swap symmetric cells, stopping halfway):

```
int n = strlen(s);
for (i = 0; i < n/2; i++) {
  char t = s[i]; s[i] = s[n-1-i]; s[n-1-i] = t;
}
```

Validating a telephone number (must contain exactly 9 digits, as for a Cameroonian mobile number):

```
int ok = (strlen(tel) == 9);
for (i = 0; ok && i < 9; i++)
  if (!isdigit(tel[i])) ok = 0;
```

WORKED EXAMPLE 5 **Vowel counter, fully worked**

Statement: write a program that reads a word (at most 29 letters) and prints the number of vowels it contains.

SOLUTION

Solution:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(void) {
  char word[30]; int i, count = 0;
  fgets(word, 30, stdin);
  word[strlen(word)-1] = '\0';
  for (i = 0; i < strlen(word); i++) {
    char c = tolower(word[i]);
    if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
  }
  printf("Vowels: %d", count);
  return 0;
```

- `}`

For the input `Yaounde`, the loop examines 7 characters and prints `Vowels: 4`.

4.6 One-dimensional arrays

DEFINITION **1D array**

A **one-dimensional array** is a block of consecutive memory cells, all of the same type, accessed by one name and an integer **index**. Declaration: `int marks[30];` reserves 30 cells of type `int`.

PROPERTY **Indices run from 0 to size −1**

For an array declared as `int a[30];`, the valid indices are $0, 1, \ldots, 29$. The element `a[30]` does not exist: the last element is `a[29]`. The size and the last index must never be confused.

Initialisation at declaration uses a brace list:

- `int marks[5] = {12, 8, 15, 10, 7};`
- `float temp[7] = {0};` (all cells set to 0)

WARNING **Out-of-bounds access is undefined behaviour**

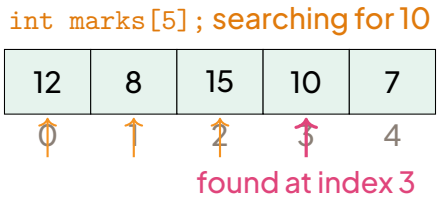
Reading or writing `a[30]` in an array of size 30 does not produce a clean error message. The program may crash, corrupt another variable, or appear to work while silently destroying data. Always keep indices inside [0, size − 1].

METHOD **Template loops for a 1D array of `n` elements**

1. Filling: `for (i = 0; i < n; i++) scanf("%d", &a[i]);`
2. Displaying: `for (i = 0; i < n; i++) printf("%d ", a[i]);`
3. Sum / average: initialise `sum = 0;` then `sum += a[i];` inside the loop; average is `(float)sum / n`.
4. Searching a value `x`: loop with a flag: `found = 0; for (i = 0; i < n; i++) if (a[i]`

`== x) { found = 1; pos = i; }`

5. **Maximum and its position:** assume `max = a[0]; posMax = 0;` then for `i` from 1: `if (a[i] > max) { max = a[i]; posMax = i; }`



Linear search: the loop inspects cells 0, 1, 2, 3 in order until the value 10 is found.

4.7 Sorting a small array: the bubble sort

Sorting means rearranging the elements in ascending (or descending) order. The **bubble sort** repeatedly compares neighbouring elements and swaps them if they are in the wrong order; after each full pass, the largest remaining value "bubbles up" to its final position.

```
for (pass = 0; pass < n-1; pass++)
  for (i = 0; i < n-1-pass; i++)
    if (a[i] > a[i+1]) {
      int t = a[i]; a[i] = a[i+1]; a[i+1] = t;
    }
```

EXEMPLE **Trace of the first pass on** {12, 8, 15, 10}

Comparison	Swap?	Array after step	Comment
`a[0]` vs `a[1]`: 12, 8	yes	8, 12, 15, 10	12 moves right
`a[1]` vs `a[2]`: 12, 15	no	8, 12, 15, 10	already ordered
`a[2]` vs `a[3]`: 15, 10	yes	8, 12, 10, 15	15 in final place

After pass 1, the maximum (15) is at the last index. Two more passes give 8, 10, 12, 15.

4.8 Arrays and strings together: an array of names

A list of student names is an array of strings, which in C is a two-dimensional array of `char`:

- `char students[45][30];` (45 names, at most 29 letters each)

- `strcpy(students[0], "Amina");`
- `printf("%s", students[0]);`

Each row `students[i]` is itself a complete string with its own `'\0'`, so all the string functions apply to it directly. A loop over `i` can then, for example, print the class list or search for a name with `strcmp`.

4.9 Debugging arrays and strings

WARNING **The three most frequent bugs**

1. Out-of-bounds index: writing `for (i = 0; i <= n; i++)` visits `a[n]`, one cell too far. Use `i < n`.
2. Missing null terminator: building a string cell by cell and forgetting `s[n] = '\0';` makes `printf("%s", s)` print garbage beyond the text.
3. Confusing length and last index: for a string of length `n = strlen(s)`, the last character is `s[n-1]` and `s[n]` is the terminator itself.

EXERCISE 4 **Class marks analysis**

Write a C program that reads the marks (integers out of 20) of 30 students into an array, then displays the average, the highest mark and its position in the array.

SOLUTION **Exercise 1**

```
int marks[30], i, sum = 0, max, posMax;
for (i = 0; i < 30; i++) scanf("%d", &marks[i]);
max = marks[0]; posMax = 0;
for (i = 0; i < 30; i++) {
  sum += marks[i];
  if (marks[i] > max) { max = marks[i]; posMax = i; }
}
printf("Average: %.2f\n", sum / 30.0);
printf("Best: %d at index %d", max, posMax);
```

Note the initialisation `max = marks[0]`: starting with `max = 0` would fail if all marks were negative (not possible here, but a dangerous habit).

KEY POINTS **Key points**

- A `char` stores one ASCII-coded character; `<ctype.h>` tests and converts characters.
- A string is an array of `char` ended by `'\0'`; use `fgets` for safe input and `strlen`, `strcpy`, `strcat`, `strcmp` from `<string.h>`.
- Never compare strings with `==`; use `strcmp(a, b) == 0`.
- An array `a[n]` has valid indices 0 to $n-1$; out-of-bounds access is undefined behaviour.
- Filling, displaying, summing, searching, finding the maximum and bubble sorting are all done with simple `for` loops over the indices.

DID YOU KNOW? **Why does C trust the programmer?**

C was designed in the early 1970s to write operating systems, where speed was everything, so it performs no automatic bounds checking. Many famous security attacks (buffer overflows) exploit exactly the bugs studied in this section. Modern systems such as the mobile money platforms used daily in Cameroon are still partly written in C — which is why disciplined array handling is a professional skill, not just an exam topic.

5 TWO-DIMENSIONAL ARRAYS, RECORDS AND LISTS

In the previous section we worked with **one-dimensional arrays**: a single row of values, such as the marks of one student in three subjects, `float marks[3];`. But a real class at Government High School is not one student — it is forty students, each with several marks. We need a table of values: rows for students, columns for subjects. That is exactly what a **two-dimensional array** provides. We then go further: a student is not only marks, but also a name, an age, an average — data of different types that belong together. C groups them with a **record**, declared using `struct`. This section combines both ideas to build a complete class list, one of the most frequent GCE practical situations.

5.1 Two-dimensional arrays: tables of values

Intuition.

Think of the mark sheet used by a form master: each row is a student, each column is a subject (Mathematics, Physics, Computer Science). To locate one mark you need two coordinates: the

row number and the column number. A 2D array generalises the 1D array in exactly this way.

DEFINITION **Two-dimensional array**

A **two-dimensional array** is a collection of elements of the same type, organised in rows and columns, and accessed by two indices. It is declared as:

```
type name[nbRows][nbColumns];
```

For example, the marks of 40 students in 3 subjects:

```
float notes[40][3];
```

The element `notes[i][j]` is the mark of student `i` in subject `j`. Indices start at 0, so valid rows are `0` to `39` and valid columns `0` to `2`.

WARNING **Row first, column second**

In `notes[i][j]`, the first index is the row and the second index is the column. Writing `notes[2][0]` when you mean `notes[0][2]` silently reads the wrong cell — the program compiles but gives wrong results. Remember: row, then column, like "line, then seat" in a classroom.

	col 0	col 1	col 2
row 0	12	9	14
row 1	8	15	11
row 2	13	7	16
row 3	10	12	9

`notes[4][3]` — diagonal cells shaded

A 2D array `notes[4][3]`: 4 rows (students) and 3 columns (subjects). The shaded cells `notes[i][i]` form the main diagonal.

5.2 Traversing a 2D array with nested loops

Because each cell has two indices, we need two nested loops: the outer loop fixes the row, the inner loop scans the columns of that row.

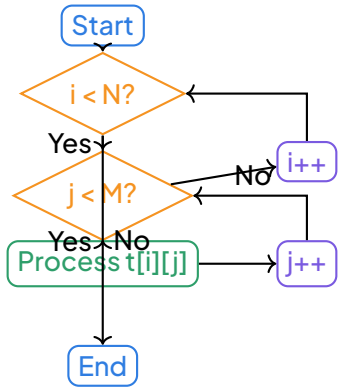
DEFINITION **Row-major traversal**

A **row-major traversal** visits the array row by row: for each row `i`, all columns `j` are visited from left to right before moving to the next row. This is the natural order for reading or printing a table.

METHOD **Nested-loop template for a 2D array**

To process every cell of `float t[N][M];`:

1. Outer loop over rows: `for (i = 0; i < N; i++)`
2. Inner loop over columns: `for (j = 0; j < M; j++)`
3. Body: process `t[i][j]` (read it, print it, accumulate it).
4. To print a table, put a newline `\n` after the inner loop, so each row appears on its own line.



Flowchart of row-major nested loop traversal.

EXEMPLE **Filling and displaying a marks table**

```
float notes[40][3]; int i, j;
for (i = 0; i < 40; i++) {
  printf("Student %d\n", i + 1);
  for (j = 0; j < 3; j++) {
    printf(" Mark subject %d: ", j + 1);
```

```
      scanf("%f", &notes[i][j]);
    }
  }
  for (i = 0; i < 40; i++) {
    for (j = 0; j < 3; j++)
      printf("%6.1f ", notes[i][j]);
    printf("\n");
  }
```

Note the address-of operator `&` in `scanf`, and the newline printed between rows, not after every mark.

5.3 Classic 2D processing: totals, averages, searching

Three processing patterns appear constantly at GCE level.

1. Row sums: total per student. For each row `i`, accumulate the columns:

```
for (i = 0; i < 40; i++) {
  float total = 0;
  for (j = 0; j < 3; j++) total = total + notes[i][j];
  printf("Student %d: total = %.1f\n", i+1, total);
}
```

The accumulator `total` is declared and reset to 0 inside the outer loop — one total per student.

2. Column averages: class average per subject. Swap the roles: for each column `j`, accumulate down the rows:

```
for (j = 0; j < 3; j++) {
  float sum = 0;
  for (i = 0; i < 40; i++) sum = sum + notes[i][j];
  printf("Subject %d average = %.2f\n", j+1, sum/40);
}
```

3. Searching in a matrix. Scan all cells and stop when found:

```
int found = 0;
for (i = 0; i < 40 && !found; i++)
```

```
    for (j = 0; j < 3 && !found; j++)
      if (notes[i][j] == target) found = 1;
```

WARNING **Wrong loop order**

If you want a total per student (per row), the `j` loop must be inside and the accumulator reset for each i. Putting `total = 0;` before both loops gives one single grand total; swapping the loops gives totals per subject instead. Always ask: "what varies in the inner loop?"

EXEMPLE **Trace table for row totals (3 students, 2 subjects)**

Consider `float notes[3][2] = {{10,12},{8,14},{11,9}};`. The trace of the row-total algorithm:

i	j	notes[i][j]	total (after add)	Output
0	0	10	10	
0	1	12	22	Student 1: 22.0
1	0	8	8	
1	1	14	22	Student 2: 22.0
2	0	11	11	
2	1	9	20	Student 3: 20.0

5.4 Matrices at GCE level

A **matrix** is a square (or rectangular) 2D array of numbers, as in Further Mathematics. Two classic tasks:

Sum of the main diagonal. The diagonal cells are those with `i == j`, so a single loop suffices for an $n \times n$ matrix:

```
float s = 0;
for (i = 0; i < n; i++) s = s + a[i][i];
```

Transpose. The transpose swaps rows and columns: $b_{ji} = a_{ij}$.

```
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    b[j][i] = a[i][j];
```

DID YOU KNOW? **GCE tip**

GCE practical and written questions very often present a marks table and ask for totals per row (per candidate) and averages per column (per subject), sometimes followed by ranking. Master the two nested-loop patterns above and you can answer all of them.

5.5 Records: grouping data of different types with struct

Intuition.

An array stores many values of the same type. But a student is described by a name (string), an age (integer) and an average (float). Storing these in three separate parallel arrays is fragile: deleting a student means editing three arrays consistently. C's solution is the **record**.

DEFINITION **Record and field**

A **record** (in C, a **structure**) is a composite data type that groups several values, called **fields**, which may have different types, under one name. It is defined with `struct`:

```
struct Student {
  char name[30];
  int age;
  float average;
};
```

Each field is accessed with the dot operator: if `s` is a `struct Student`, then `s.name`, `s.age` and `s.average` are its fields.

WARNING **The semicolon after the brace**

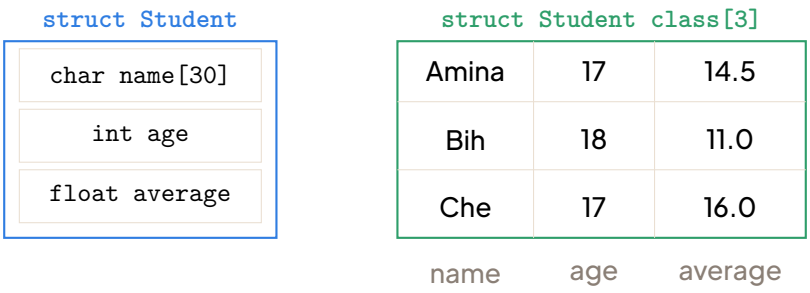
A `struct` definition must end with a semicolon after the closing brace: `};`. Forgetting it is one of the most common C errors, and the compiler's error message usually points to the next line, which confuses beginners. Check the semicolon first.

Declaring variables and using fields.

```
struct Student s1, s2;
strcpy(s1.name, "Amina");
s1.age = 17;
s1.average = 14.5;
```

```
s2 = s1;   /* copies ALL fields at once */
```

Two important details: since `name` is a string, we copy text into it with `strcpy` (from `string.h`), never with =. But assignment between two whole records with = is legal and copies every field — a very convenient feature.



Left: the fields of `struct Student`. Right: an array of records `class[3]` seen as a table — each row is one record.

5.6 Arrays of records: the class list

The real power comes from combining the two ideas: an array of records.

```
struct Student class[40];
```

Then `class[i]` is the record of student `i`, and `class[i].average` is that student's average. Note the order: index first, then dot, then field.

Filling the list:

```
for (i = 0; i < n; i++) {
  printf("Name: "); scanf("%s", class[i].name);
  printf("Age: "); scanf("%d", &class[i].age);
  printf("Average: "); scanf("%f", &class[i].average);
}
```

Observe: `class[i].name` needs no `&` (a string is already an address), but `&class[i].age` and `&class[i].avera` do.

Counting records satisfying a condition (students with average at least 10):

```
int count = 0;
for (i = 0; i < n; i++)
  if (class[i].average >= 10) count++;
```

Finding the best student (classic maximum search keeping an index):

```
int best = 0;
for (i = 1; i < n; i++)
```

```
    if (class[i].average > class[best].average) best = i;
printf("Best: %s (%.2f)\n", class[best].name, class[best].average);
```

Adding a record to a list that currently holds `n` students: simply fill `class[n]` and increment `n` — after checking that `n < 40`, otherwise the array overflows.

METHOD **Processing an array of records**

To solve a typical GCE problem on a class list:

1. Define the `struct` with all required fields (name, age, marks, etc.).
2. Declare the array with a maximum size (e.g., `struct Student class[100];`).
3. Read the actual number of students `n` (validate `n <= 100`).
4. Use a single loop `for (i=0; i<n; i++)` to:
   - read each record's fields (use `strcpy` for strings, `&` for numbers);
   - accumulate sums for averages;
   - track the index of the best/worst record.
5. After the loop, compute and display results using the stored index.

WORKED EXAMPLE 6 **Class list: average and best student**

A class has n students ($n \le 40$). Write a C fragment that reads each student's name and average, then displays the class average and the name of the best student.

SOLUTION

```
struct Student { char name[30]; float average; };
struct Student class[40];
int n, i, best = 0; float sum = 0;
scanf("%d", &n);
for (i = 0; i < n; i++) {
  scanf("%s", class[i].name);
  scanf("%f", &class[i].average);
  sum = sum + class[i].average;
  if (class[i].average > class[best].average) best = i;
```

```
    }
    printf("Class average: %.2f\n", sum / n);
    printf("Best student: %s\n", class[best].name);
```

Key points: `best` stores an index, initialised to 0 before the loop; the two results (sum and best) are computed in a single pass through the list.

5.7 Nested structures and debugging (GCE enrichment)

A field of a record can itself be a record. For example, a date of birth inside a student:

```
struct Date { int day, month, year; };
struct Student {
  char name[30];
  struct Date birth;
  float average;
};
```

Access chains the dots: `s.birth.year = 2008;`. This is enrichment beyond the strict GCE core, but it shows how records model real data naturally.

WARNING **Debugging records and 2D arrays**

The four most frequent errors, and their symptoms:

1. Swapped indices: `notes[j][i]` instead of `notes[i][j]` — compiles, but reads the transpose; check which index is the row.
2. Missing semicolon after `struct { ... }` — compiler error reported on the following line.
3. Mixing field types: reading an `int` field with `"%f"` or assigning a string with = instead of `strcpy` — garbage values or crashes.
4. Wrong accumulator position: resetting a total outside the row loop — one grand total instead of per-student totals.

EXERCISE 5 **Marks table processing**

Declare `float notes[30][4];` for 30 students and 4 subjects. Write C fragments to: (a) fill the table; (b) display each student's total; (c) display the class average per subject; (d)

display the highest mark in the table with its row and column.

SOLUTION **Exercise 1**

```
float notes[30][4]; int i, j;
/* (a) */ for (i=0;i<30;i++) for (j=0;j<4;j++) scanf("%f",&notes[i][j]);
/* (b) */ for (i=0;i<30;i++) { float t=0;
  for (j=0;j<4;j++) t+=notes[i][j]; printf("%.1f\n",t); }
/* (c) */ for (j=0;j<4;j++) { float s=0;
  for (i=0;i<30;i++) s+=notes[i][j]; printf("%.2f\n",s/30); }
/* (d) */ float max=notes[0][0]; int ri=0,ci=0;
for (i=0;i<30;i++) for (j=0;j<4;j++)
  if (notes[i][j]>max) { max=notes[i][j]; ri=i; ci=j; }
```

KEY POINTS **Key points**

- A 2D array `t[N][M]` is a table: first index = row, second = column; both start at 0.
- Process it with nested loops: outer loop over rows, inner over columns; reset row accumulators inside the outer loop.
- Diagonal cells satisfy `i == j`; the transpose is `b[j][i] = a[i][j]`.
- A `struct` groups fields of different types; access them with the dot operator; end the definition with a semicolon.
- An array of records models a class list; search by keeping the index of the best record; copy strings with `strcpy`, whole records with `=`.

6 SUBROUTINES, DEBUGGING AND INTEGRATION

In the previous section we learnt how to organise data: two-dimensional arrays for tables, records (`struct`) for grouping related fields, and simple lists. But data alone does not make a program. A real program — a school results manager for a college in Bamenda, a stock program for a shop in Douala — also needs its instructions to be organised. A program of 500 lines written as one

single `main` function is like a house with no rooms: everything is in one hall, impossible to navigate. In this final section we organise the actions of a program using **subroutines** (functions in C), we learn a professional method for **debugging**, and we finish with an **integration workshop** that combines everything learnt in this chapter.

6.1 Why decompose a program?

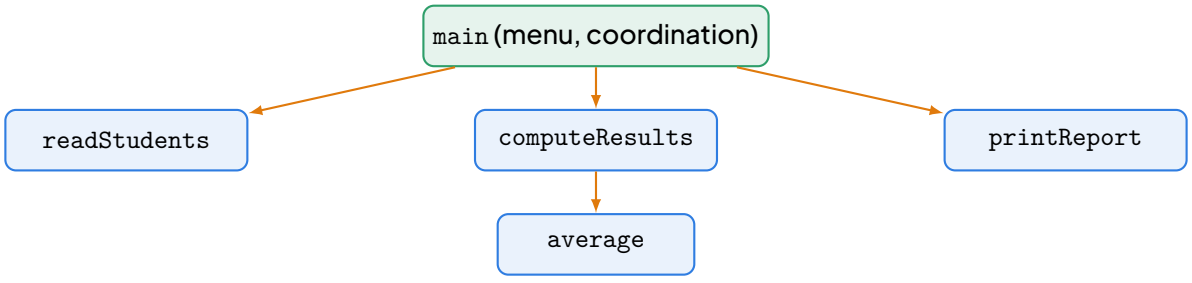
Observation. Imagine cooking achu for twenty guests while trying, at the same moment, to pound the cocoyams, prepare the soup, wash the dishes and receive visitors — all in one continuous action with no separation. You would quickly lose track. Cooks naturally decompose: one task for the cocoyams, one for the soup, one for serving. Programmers do exactly the same: this approach is called **modular programming** or top-down design — break a big problem into smaller sub-problems, solve each one separately, then assemble the solutions.

DEFINITION **Subroutine / function**

A **subroutine** is a named, self-contained block of instructions that performs one well-defined task and can be called (executed) from other parts of the program. In C, every subroutine is called a **function**. A program is a collection of functions, one of which must be `main`, where execution always begins.

Decomposing a monolithic program (everything in `main`) into functions gives three decisive advantages:

- **Readability.** Each function has a name that says what it does: `readMarks`, `computeAverage`, `printReport`. Reading `main` becomes reading a summary.
- **Reuse.** A function written once can be called many times, and even copied into another program. No duplicated code.
- **Easier debugging and testing.** If the average is wrong, you only inspect `computeAverage`, not the whole program. Each function can be tested separately.



Top-down structure chart of the integration project: `main` delegates the work to specialised functions, which may themselves call simpler functions.

6.2 Defining and calling a function

DEFINITION **Function definition, parameter, argument, return value, prototype**

- A **function definition** has the form: return type, name, parameter list in parentheses, then a body in braces.
- A **parameter** is a variable declared in the function header that receives a value when the function is called.
- An **argument** is the actual value (or variable) supplied by the caller at the point of the call. Arguments are matched to parameters in order, and their types must be compatible.
- The **return value** is the single value the function sends back to its caller, using the `return` statement. A function that returns nothing has return type `void`; such a function is sometimes called a procedure.
- A **prototype** (declaration) is the function header followed by a semicolon, placed before `main` (or in a header file), so the compiler knows the function exists before it is used.

EXEMPLE **A first function**

```
#include <stdio.h>

float average(float a, float b); /* prototype */

int main(void) {

    float m1 = 12.5, m2 = 15.0, m;

    m = average(m1, m2); /* call: m1, m2 are arguments */

    printf("Mean = %.2f\n", m);

    return 0;

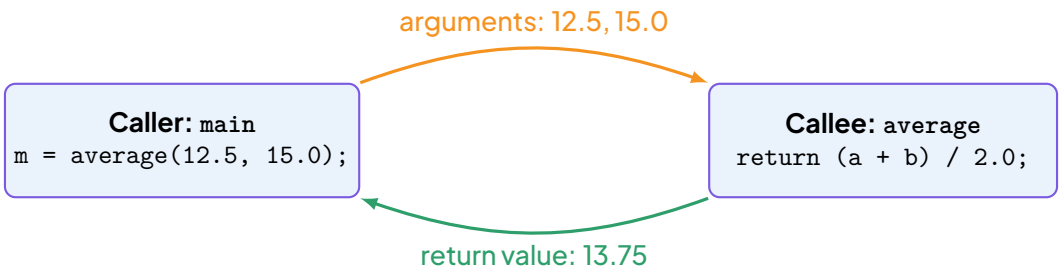
}

float average(float a, float b) { /* definition */

    return (a + b) / 2.0; /* return value */

}
```

Here `a` and `b` are the parameters; `m1` and `m2` are the arguments. The call `average(m1, m2)` copies `12.5` into `a` and `15.0` into `b`, executes the body, and sends `13.75` back into `m`. The program prints `Mean = 13.75`.



The function call mechanism: the caller passes arguments, the callee computes and returns one value, then execution continues in the caller just after the call.

WARNING **Use before declaration**

Calling a function that has not been declared (no prototype before `main`) produces compiler errors or, worse, silently wrong behaviour because the compiler guesses the types. Rule: always write the prototype before `main`, or define the function itself before `main`. Also check that the number, order and types of the arguments match the prototype exactly.

6.3 Passing data to functions: by value, arrays, structures

PROPERTY **Passing mechanism in C (examinable)**

In C, simple variables (`int`, `float`, `char`) are passed **by value**: the function receives a copy. Modifying the parameter inside the function does not change the caller's variable. In contrast, an **array** is passed **by reference**: the function receives access to the original array, so modifications inside the function are visible to the caller. A `struct` is passed by value (a copy), unless one passes its address.

EXEMPLE **By value versus by reference**

```
void tryChange(int x) { x = 99; } /* changes only the copy */

void fillMarks(int t[], int n) { /* t is the ORIGINAL array */
    int i;
    for (i = 0; i < n; i++) scanf("%d", &t[i]);
}
```

```
/* in main: */
int a = 5, marks[30];
tryChange(a); /* a is still 5 */
fillMarks(marks, 30); /* marks is really filled */
```

Note the good habit visible in `fillMarks`: an array parameter is always accompanied by an `int` parameter giving its effective size, because a function cannot know by itself how many elements the array really contains.

WARNING **The classic trap**

A very common GCE error: writing `void doubleIt(int n) { n = 2 * n; }` and expecting the caller's variable to double. It does not — the function worked on a copy. To "return" several results, either return a `struct`, or modify an array passed as parameter, or (beyond the syllabus) use pointers.

A record is passed like any simple type:

```
struct Student { char name[30]; int age; float mean; };
void display(struct Student s) {
    printf("%s, %d years, mean %.2f\n", s.name, s.age, s.mean);
}
```

6.4 Local variables, global variables, scope and lifetime

DEFINITION **Scope and lifetime**

The **scope** of a variable is the region of the program where its name is visible. A variable declared inside a function (including parameters) is **local**: it exists only while that function runs (its lifetime) and is invisible elsewhere. A variable declared outside all functions is **global**: visible from its declaration to the end of the file, for the whole run of the program. When a local variable has the same name as a global one, the local one hides the global one inside that function.

Global variables are tempting ("every function can see them!") but dangerous: any function can change them, so when a wrong value appears you cannot tell who changed it. They also make functions non-reusable, because the function depends on hidden external data.

WORKED EXAMPLE 7 **Predicting the output: local versus global**

Statement: what does this program print?

```
#include <stdio.h>

int x = 1; /* global */

void f(void) { int x = 10; x = x + 1; printf("f: %d\n", x); }

int main(void) { f(); x = x + 1; printf("main: %d\n", x); return 0; }
```

SOLUTION

Solution. Inside `f`, the local `x` hides the global one: `x` becomes 10 + 1 = 11, so `f` prints `f: 11`. The global `x` is untouched (still 1). Back in `main`, `x = x + 1` uses the global `x`, which becomes 2: `main` prints `main: 2`. Output:

```
f: 11

main: 2
```

KEY POINTS **Good practice**

Prefer local variables and pass data through parameters and return values. Reserve global variables for true constants (e.g. `#define MAX 30` or `const int MAX = 30;`).

6.5 A systematic debugging method

DEFINITION **Debugging**

Debugging is the process of finding and correcting errors (bugs) in a program. Errors fall into three classes: syntax errors (the program does not compile), logic errors (it compiles and runs but gives wrong results) and runtime errors (it crashes or misbehaves during execution).

Type	Symptom	Example
Syntax	Compiler refuses; error messages with line numbers	Missing semicolon: `int a = 5`
Logic	Runs, but wrong output	Average computed as `(a+b)/3` instead of `/2`
Runtime	Crashes or freezes during execution	Division by zero; reading `t[30]` in an array of size 30

METHOD **The 4-step debugging strategy**

1. Observe / reproduce. Find input data that makes the bug appear every time. A bug you cannot reproduce cannot be fixed.
2. Locate. Read compiler warnings carefully (start with the first one — later messages are often consequences of it). Insert trace prints such as `printf("DEBUG: i=%d sum=%d\n", i, sum);` to follow the values, and narrow down the faulty function.
3. Correct. Fix the cause, not the symptom. Change one thing at a time.
4. Re-test. Re-run the failing case and the cases that previously worked, to check the fix broke nothing.

Practical enrichment: a debugger (e.g. `gdb`, or the one built into Code::Blocks) lets you set breakpoints, execute step by step and watch variables — a powerful way to locate logic errors without scattering `printf` calls.

WORKED EXAMPLE 8 **Three bugs, three corrections**

Statement: each fragment below contains one error. Classify it and correct it.

(a) `int x = 10 printf("%d", x);`

(b) `float mean = (eng + math) / 2; /* eng, math are int */`

(c) `int t[10]; ... t[10] = 5;`

SOLUTION

Solution.

- (a) Syntax error: missing semicolon after `10`. Correct: `int x = 10; printf("%d", x);`
- (b) Logic error: integer division truncates, e.g. (13 + 14)/2 gives 13 not 13.5. Correct: `float mean = (eng + math) / 2.0;`
- (c) Runtime error: valid indices are `0` to `9`; `t[10]` is out of bounds. Correct: use `t[9]` (and loops with condition `i < 10`).

6.6 Testing and documenting a program

Debugging removes bugs you have seen; testing hunts for bugs you have not seen. Choose test data in three families:

- **Normal cases**: typical values, e.g. marks 12, 15, 8.
- **Boundary cases**: extreme valid values, e.g. mark 0 and 20, an empty list, one single student.

- **Invalid cases**: data the program should reject gracefully, e.g. mark -5 or 45, a letter instead of a number.

Documentation. Comments explain why, not what. A header comment before each function (purpose, parameters, return value) and a few comments at difficult steps are enough. Example: `/* Returns the average of the n marks in t; n must be > 0 */`.

6.7 Integration workshop: a complete modular program

We now assemble everything from this chapter — environment, basic instructions, selection, iteration, strings, 1D arrays, records and functions — into one project: a small **class results manager**.

METHOD **Building the program top-down**

1. Specify. The program stores up to 30 students (name, mark), offers a menu: 1) enter students, 2) display report with each student's verdict, 3) show class average and best student, 0) quit.
2. Decompose (see the structure chart above): `main` runs the menu loop; `readStudents`, `printReport`, `classAverage`, `bestStudent` do the work.
3. Write prototypes, then `main`, then the functions, compiling after each function.
4. Test with normal, boundary and invalid data; document with comments.

EXEMPLE **The integration project (skeleton with key functions)**

```
#include <stdio.h>

#include <string.h>

#define MAX 30

struct Student { char name[30]; float mark; };

int readStudents(struct Student s[]);

void printReport(struct Student s[], int n);

float classAverage(struct Student s[], int n);

int bestStudent(struct Student s[], int n);

int main(void) {

    struct Student cls[MAX]; int n = 0, choice;

    do {
```

```
        printf("1-Enter 2-Report 3-Stats 0-Quit: ");
        scanf("%d", &choice);
        if (choice == 1) n = readStudents(cls);
        else if (choice == 2) printReport(cls, n);
        else if (choice == 3 && n > 0) {
            int b = bestStudent(cls, n);
            printf("Average: %.2f\n", classAverage(cls, n));
            printf("Best: %s (%.2f)\n", cls[b].name, cls[b].mark);
        }
    } while (choice != 0);
    return 0;
}
float classAverage(struct Student s[], int n) {
    float sum = 0; int i;
    for (i = 0; i < n; i++) sum = sum + s[i].mark;
    return sum / n;
}
int bestStudent(struct Student s[], int n) {
    int i, b = 0;
    for (i = 1; i < n; i++) if (s[i].mark > s[b].mark) b = i;
    return b; /* returns the INDEX of the best */
}
```

Notice how `main` reads like a summary, how the array `cls` and its size `n` travel together as parameters, and how each function does exactly one job. `readStudents` and `printReport` (a validation loop and a loop with `if (mark >= 10)` for the verdict) are left as exercise.

EXERCISE 6 **Complete the project**

1. Write `readStudents`: ask how many students (validate: between 1 and `MAX`), then read

each name and mark (validate: 0 ≤ mark ≤ 20), and return the number of students.

2. Write `printReport`: display a table of names, marks and the verdict "Admitted" (mark ≥ 10) or "Failed".
3. Design a test table for the whole program with at least two normal, two boundary and two invalid cases.

SOLUTION **Exercise: Complete the project**

```
int readStudents(struct Student s[]) {
    int n, i;
    do { printf("How many (1-%d)? ", MAX); scanf("%d", &n);
    } while (n < 1 || n > MAX);
    for (i = 0; i < n; i++) {
       printf("Name: "); scanf("%s", s[i].name);
       do { printf("Mark (0-20): "); scanf("%f", &s[i].mark);
       } while (s[i].mark < 0 || s[i].mark > 20);
    }
    return n;
}
void printReport(struct Student s[], int n) {
    int i;
    for (i = 0; i < n; i++)
       printf("%-20s %6.2f %s\n", s[i].name, s[i].mark,
              s[i].mark >= 10 ? "Admitted" : "Failed");
}
```

Test table (examples): normal — 3 students with marks 12, 7, 16; boundary — 1 single student with mark 0, and a full class of 30; invalid — number of students -3 or 50, mark 25 (must be re-asked).

WARNING **GCE tip — what the examiner expects**

At the Advanced Level you must be able to: (i) write a function from a given specification (correct header, correct return type, correct use of parameters); (ii) identify the scope of each variable in a given program and predict the output when local and global variables share names; (iii) classify an error as syntax, logic or runtime and propose a correction; (iv) trace a function call by hand, showing argument values and the returned value. Practise writing short functions on paper — without a compiler, exactly as in the examination room.

KEY POINTS **Key points of the section**

- Decompose every non-trivial program into functions: readability, reuse, easier debugging.
- A function has a return type, a name, parameters and a body; declare its prototype before `main`.
- Simple variables are passed by value (a copy); arrays are passed by reference (the original).
- Prefer local variables; limit global ones; a local variable hides a global variable of the same name.
- Debug methodically: observe, locate, correct, re-test — and test with normal, boundary and invalid data.

7 INTEGRATION ACTIVITY

7.1 Aims of the activity

This integration activity closes the chapter by bringing together, in one single project, every construct studied in Lessons 18 to 26. By the end of the activity, each pair of students should be able to:

- set up and use a C programming environment (editor, compiler such as `gcc`, and terminal) to write, compile and run a program;
- declare and manipulate variables of the basic types (`int`, `float`, `char`) and use basic instructions: assignment, `scanf`, `printf`;
- implement selection with `if`, `if...else` and `else...if` ladders, and iteration with `for`, `while` and `do...while`;
- handle strings and characters using the library functions `strcpy`, `strcmp`, `strlen`;
- implement one-dimensional and two-dimensional arrays traversed by loops;
- group related data into records (`struct`) and implement a list as an array of records;
- decompose a problem into subroutines (functions with parameters and return values);
- test a program with normal, boundary and invalid data, and debug it methodically.

KEY POINTS **Skills assessed**

Correctness of the program, modularity (one function per task), quality of input validation, quality of the test plan, and clarity of the documentation (comments).

7.2 Situation

The "College Report Card" of Government Bilingual High School Bamenda.

The principal of a fictitious college in Bamenda, GBHS Up-Station, wants to computerise the end-of-term summaries of the single Upper Sixth C class. At present, the class master computes totals and averages by hand on paper sheets, which is slow and error-prone. You are asked, working in pairs, to write a menu-driven C program that does the job.

The class has at most **40 students**. For each student the program stores:

- the name (a string of at most 30 characters);
- the marks out of 20 in three subjects: Mathematics, Computer Science and Physics.

The program must display the following menu and repeat until the user chooses to quit:

```
GBHS UP-STATION – REPORT CARD SYSTEM
1. Enter students and marks
2. Compute totals and averages
3. Display full table with grades
4. Search a student by name
5. Class statistics (best student, subject averages)
0. Quit
```

The rules fixed by the principal are:

- every mark must be between 0 and 20 inclusive; any other value is rejected and re-asked;
- the total is the sum of the three marks; the average is the total divided by 3;
- the grade is assigned as follows: average ≥ 16: “Excellent”; ≥ 14: “Very Good”; ≥ 12: “Good”; ≥ 10: “Pass”; below 10: “Fail”;
- the best student is the one with the highest total; the class average per subject is the mean of the marks of all entered students in that subject.

Test data provided by the class master (5 students, to be typed first):

Name	Maths	CompSc	Physics
Acha Brenda	18	20	16
Ngong Peter	12	9	11
Fru John	0	20	10
Tabi Grace	14	13	15
Wirba Collins	8	7	9

Note that the data deliberately contain the boundary marks 0 and 20. During the defence, the teacher will also type invalid marks (e.g. -3 or 25) to check your validation, and will have introduced **three deliberate bugs** into a copy of your code that you must find and correct.

7.3 Tasks

1. **Design.** On paper, identify the data structures needed. Justify the choice of a `struct` for a student, and decide whether the three marks are stored as three fields or as a small array inside the record. Draw the modular structure: list the functions you will write, with their parameters and return types.
2. **Skeleton and menu.** Write the `main` function implementing the menu with a `do...while` loop and a `switch` (or else-if ladder). Compile and test that the menu repeats and quits correctly before writing anything else.
3. **Data entry with validation.** Write the function `enterStudents` which asks for the number of students (between 1 and 40), then for each student the name and the three marks. Each mark is read inside a loop that rejects any value outside [0, 20].

4. **Computation.** Write `computeAll` which, using loops, fills in the total and the average of every student. Write a function `grade` that receives an average and returns the grade as a string, using an else-if ladder.
5. **Display.** Write `displayTable` which prints a neat table (name, three marks, total, average with two decimals, grade) using the field-width options of `printf`.
6. **Search.** Write `searchStudent` which asks for a name and, using `strcmp` inside a loop, displays the full record of the student or the message "Student not found".
7. **Statistics.** Write `statistics` which finds and displays the best student (highest total) and the class average in each of the three subjects.
8. **Testing.** Build a test table: normal data, boundary data (mark = 0, mark = 20, one single student, 40 students), invalid data (mark -3, mark 25, number of students = 0 or 41). Record the expected and obtained results.
9. **Debugging.** The teacher's copy of your program contains three bugs (for example: `=` instead of `==` in a test, a loop starting at index 1, an integer division for the average). Find them, correct them, and for each one write: symptom, cause, correction.
10. **Documentation.** Add a header comment (authors, date, purpose) and a comment above each function. Write half a page explaining how to compile and run the program.

WARNING **Common traps in this project**

- Comparing strings with `==` instead of `strcmp`: this compares addresses, not contents.
- Reading a name with spaces using `scanf("%s", ...)`: it stops at the first space. Use `fgets` or accept names without spaces.
- Computing the average as `total / 3` with `total` an `int`: the result is truncated. Cast to `float`: `total / 3.0f`.
- Forgetting that valid array indices run from 0 to `n - 1`.

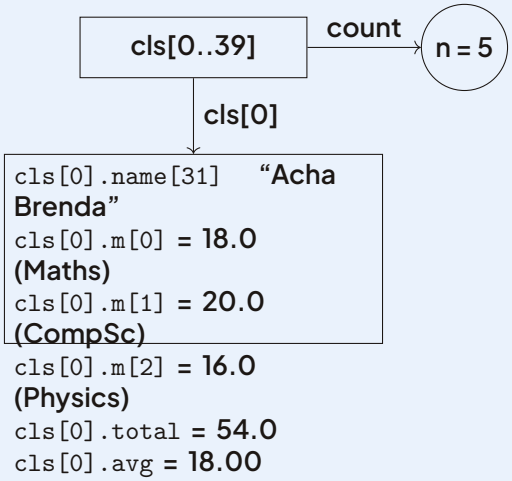
7.4 Model answer

1. Design. A student is a record; the class is a list implemented as an array of records plus a counter `n`. The three marks are stored as an array `m[3]` inside the record so that loops can process them uniformly.

- `struct Student { char name[31]; float m[3]; float total; float avg; };`
- `struct Student cls[40]; int n = 0;`
- Functions: `enterStudents`, `computeAll`, `grade`, `displayTable`, `searchStudent`, `statistics`.

EXPERIMENT **Data structure diagram**

The following diagram shows the memory layout for the class list:



2–3. Menu and validated data entry.

METHOD **Building the menu skeleton**

1. Declare global structures and counter `n`.
2. Write `main` with a `do...while` loop.
3. Inside the loop: display menu, read `choice`, dispatch with `switch`.
4. Compile and run: the menu should appear, accept a choice, and repeat until 0 is entered.

```
int main(void) {
  int choice;
  do {
    printf("1 Enter 2 Compute 3 Table 4 Search 5 Stats 0 Quit\n");
    scanf("%d", &choice);
    switch (choice) {
      case 1: enterStudents(); break;
      case 2: computeAll(); break;
      case 3: displayTable(); break;
      case 4: searchStudent(); break;
```

```
      case 5: statistics(); break;
      case 0: printf("Goodbye!\n"); break;
      default: printf("Invalid choice\n");
    }
  } while (choice != 0);
  return 0;
}
void enterStudents(void) {
  int i, j; float mark;
  do {
    printf("Number of students (1..40): ");
    scanf("%d", &n);
  } while (n < 1 ||n > 40);
  for (i = 0; i < n; i++) {
    printf("Name: "); scanf("%30s", cls[i].name);
    for (j = 0; j < 3; j++) {
      do {
        printf("Mark subject %d (0..20): ", j + 1);
        scanf("%f", &mark);
      } while (mark < 0 ||mark > 20);
      cls[i].m[j] = mark;
    }
  }
}
```

4. Computation and grade.

```
void computeAll(void) {
  int i, j;
  for (i = 0; i < n; i++) {
    cls[i].total = 0;
    for (j = 0; j < 3; j++)
```

```
      cls[i].total = cls[i].total + cls[i].m[j];
    cls[i].avg = cls[i].total / 3.0f;
  }
}
const char *grade(float avg) {
  if (avg >= 16) return "Excellent";
  else if (avg >= 14) return "Very Good";
    else if (avg >= 12) return "Good";
    else if (avg >= 10) return "Pass";
    else return "Fail";
}
```

5. Display.

```
void displayTable(void) {
  int i, j;
  printf("%-15s %6s %6s %6s %6s %6s %s\n",
    "Name", "Maths", "Comp", "Phys", "Total", "Avg", "Grade");
  for (i = 0; i < n; i++) {
    printf("%-15s", cls[i].name);
    for (j = 0; j < 3; j++) printf(" %6.1f", cls[i].m[j]);
    printf(" %6.1f %6.2f %s\n", cls[i].total,
      cls[i].avg, grade(cls[i].avg));
  }
}
```

Expected output for the test data (after option 2):

Name	Ma	Co	Ph	Tot	Avg	Grade
Acha Brenda	18	20	16	54	18.00	Excellent
Ngong Peter	12	9	11	32	10.67	Pass
Fru John	0	20	10	30	10.00	Pass
Tabi Grace	14	13	15	42	14.00	Very Good
Wirba Collins	8	7	9	24	8.00	Fail

6. Search by name.

```
void searchStudent(void) {
  char key[31]; int i, found = 0;
  printf("Name to search: "); scanf("%30s", key);
  for (i = 0; i < n && !found; i++)
    if (strcmp(cls[i].name, key) == 0) {
      printf("Total: %.1f Avg: %.2f %s\n",
        cls[i].total, cls[i].avg, grade(cls[i].avg));
      found = 1;
    }
  if (!found) printf("Student not found\n");
}
```

7. Statistics.

```
void statistics(void) {
  int i, j, best = 0; float sum;
  for (i = 1; i < n; i++)
    if (cls[i].total > cls[best].total) best = i;
  printf("Best student: %s (%.1f)\n",
    cls[best].name, cls[best].total);
  for (j = 0; j < 3; j++) {
    sum = 0;
    for (i = 0; i < n; i++) sum = sum + cls[i].m[j];
    printf("Subject %d average: %.2f\n", j + 1, sum / n);
  }
}
```

For the test data: best student is Acha Brenda (54); subject averages are Maths = 10.40, CompSc = 13.80, Physics = 12.20.

8–9. Testing and debugging. The test table confirms that marks 0 and 20 are accepted while -3 and 25 are re-asked, and that n = 0 or 41 is rejected. Example of a debug report:

Bug	Symptom	Cause	Correction
1	Menu quits immediately	`while (choice = 0)`	`while (choice != 0)`
2	First student skipped	loop starts at `i = 1`	start at `i = 0`
3	Averages truncated	`total / 3` integer division	`total / 3.0f`

KEY POINTS **What this activity demonstrates**

A real C program is built incrementally: data structures first, then a tested menu skeleton, then one function at a time, each compiled and tested before moving on. Systematic testing with normal, boundary and invalid data, combined with a written debug report, is what distinguishes a finished product from a draft.

7.5 Compilation and execution guide

METHOD **How to compile and run the program**

1. Save the source code in a file named `report_card.c`.
2. Open a terminal in the directory containing the file.
3. Compile with the GCC compiler using the command: `gcc -Wall -Wextra -std=c11 -o report_card report_card.c`
   - `-Wall -Wextra`: enable all warnings (helps catch bugs early).
   - `-std=c11`: use the C11 standard.
   - `-o report_card`: name the executable `report_card` (or `report_card.exe` on Windows).
4. If compilation succeeds (no errors), run the program:
   - Linux/macOS: `./report_card`
   - Windows: `report_card.exe`
5. Enter the test data when prompted. Use the menu options 1–5 to verify each feature.
6. To debug, recompile with `-g` and use `gdb ./report_card` or add temporary `printf` statements.

DID YOU KNOW? **C in Cameroonian education**

The C language is the backbone of the GCE Advanced Level Computer Science practical paper. Mastery of pointers, structures, and file handling in C is essential for the Project component (Paper 3) and for university courses in engineering, computer science, and ICT at institutions like the University of Buea, University of Yaoundé I, and the National Advanced School of Engineering (ENSP).

EXERCISE 7 **Extension tasks for further practice**

1. Modify the program to save the class data to a text file (`report.txt`) and load it at startup.
2. Add a function to sort the table by average (descending) before display.
3. Implement a "Modify marks" option that lets the teacher correct a single student's marks.
4. Replace the fixed-size array `cls[40]` with dynamic memory allocation (`malloc`, `realloc`, `free`).

SOLUTION **Exercise 1 – File saving sketch**

```
void saveToFile(void) {
  FILE *f = fopen("report.txt", "w");
  if (!f) { perror("Open failed"); return; }
  fprintf(f, "%d\n", n);
  for (int i = 0; i < n; i++) {
    fprintf(f, "%s\n% .1f % .1f % .1f\n",
      cls[i].name, cls[i].m[0], cls[i].m[1], cls[i].m[2]);
  }
  fclose(f);
}
```

Loading is the reverse: `fscanf` the count, then loop reading each record. Remember to consume the newline after the count.

8 METHODS AND WORKED EXAMPLES

8.1 Methods and Worked Examples

This part trains you, step by step, on the practical skills examined at the GCE Advanced Level: writing, tracing, debugging and running C programs. Each method card is immediately followed by a fully solved GCE-style exercise.

METHOD **Writing and running your first C program**

To produce a correct, compilable C program, always follow the same skeleton:

1. Include the libraries you need: `#include <stdio.h>` for input/output, `#include <string.h>` for strings, `#include <math.h>` for mathematics.
2. Write the main function: `int main(void) { ... return 0; }`. Every C program starts executing at `main`.
3. Declare every variable with its type (`int`, `float`, `double`, `char`) before using it.
4. Prompt before reading: print a message with `printf`, then read with `scanf("%d", &x);` — never forget the `&` before the variable in `scanf`.
5. End every statement with a semicolon and every block with a closing brace `}`.
6. Compile then run: `gcc prog.c -o prog` then `./prog` (or `prog.exe` on Windows). Read the first compiler error first; later errors are often consequences.

Reflexes: format codes are `%d` (int), `%f` (float/double in `printf`), `%c` (char), `%s` (string). In `scanf`, a `double` needs `%lf`.

WORKED EXAMPLE 9 **First program: total cost of items**

A shopkeeper in Bamenda sells exercise books. Write a C program that asks for the unit price (in FCFA) of one book and the quantity bought, then displays the total amount to pay.

SOLUTION

Analysis. Two inputs (`price`, `quantity`), one computed result (`total` = price × quantity). Prices in FCFA are whole numbers, so `int` is appropriate; to be safe with large amounts we could use `long`, but `int` suffices here.
Program.

- `#include <stdio.h>`
- `int main(void) {`

```
    int price, quantity, total;
    printf("Enter unit price (FCFA): ");
    scanf("%d", &price);
    printf("Enter quantity: ");
    scanf("%d", &quantity);
    total = price * quantity;
    printf("Total to pay: %d FCFA\n", total);
    return 0;
}
```

Trace with price = 500, quantity = 12:

Step	price	quantity	total
After inputs	500	12	–
After `total = price * quantity`	500	12	6000

Output: `Total to pay: 6000 FCFA.`
Key points. The `&` in `scanf("%d", &price)` gives the address of the variable so that `scanf` can store the value there. Omitting it is the most common beginner's error and usually crashes the program. The `\n` moves the cursor to a new line after printing.

DID YOU KNOW? **C in Cameroon**

The C language, created by Dennis Ritchie at Bell Labs in the early 1970s, is the backbone of most operating systems (Linux, Windows kernel) and embedded systems. In Cameroon, C is widely used in university curricula (e.g. at the University of Buea and Yaoundé I) and in the telecom sector for programming SIM toolkit applications and USSD gateways. Mastering C gives you a solid foundation for systems programming and competitive programming contests like the ACM ICPC.

METHOD **Implementing selection with `if`, `if...else` and `switch`**

1. Identify the cases from the problem statement: how many alternatives? Are they ranges of values (use `if/else if`) or exact discrete values (use `switch`)?
2. Write conditions with relational operators (<, <=, >, >=, ==, !=) and combine them with && (AND), || (OR), ! (NOT).

3. Order the tests from the most restrictive to the most general when using `else if` chains.
4. Always brace the blocks `{ }` even for a single instruction: it prevents bugs when you later add lines.
5. For `switch`: the expression must be an integer or `char`; end each `case` with `break;`; add a `default:` case for unexpected values.

Trap: `if (x = 5)` assigns 5 to `x` and is always true; the comparison is `if (x == 5)`.

WORKED EXAMPLE 10 **Selection: GCE grade from a mark**

Write a C program that reads a mark out of 100 and displays the grade according to: 80–100: A; 70–79: B; 60–69: C; 50–59: D; 40–49: E; below 40: F. Reject any mark outside $[0, 100]$.

SOLUTION

Analysis. Six ranges plus validation: an `if / else if / else` chain is the natural choice. We test from the top downwards, so each condition only needs an upper bound (the lower bound is guaranteed by the previous failed tests).
Program.

```
#include <stdio.h>
int main(void) {
  int mark;
  printf("Enter the mark (0-100): ");
  scanf("%d", &mark);
  if (mark < 0 || mark > 100) {
    printf("Invalid mark!\n");
  } else if (mark >= 80) {
    printf("Grade A\n");
  } else if (mark >= 70) {
    printf("Grade B\n");
  } else if (mark >= 60) {
    printf("Grade C\n");
  } else if (mark >= 50) {
```

```
    printf("Grade D\n");
  } else if (mark >= 40) {
    printf("Grade E\n");
  } else {
    printf("Grade F\n");
  }
  return 0;
}
```

Trace with mark = 73: validation fails (73 is in range), 73 ≥ 80 false, 73 ≥ 70 true ⇒ prints `Grade B` and skips the rest.
Why this order works. Once `mark >= 80` has failed, we know mark < 80, so `mark >= 70` exactly captures 70 ≤ mark ≤ 79. Writing the tests in the wrong order (e.g. testing `mark >= 40` first) would classify every pass mark as E — a classic logical error that compiles perfectly but gives wrong results.

METHOD **Implementing iteration: choosing the right loop**

1. Count-controlled repetition (number of passes known in advance): use `for (i = 0; i < n; i++) { ... }`.
2. Condition-controlled repetition (stop when an event occurs, maybe zero passes): use `while (condition) { ... }`.
3. Repeat at least once (e.g. input validation): use `do { ... } while (condition);` — note the final semicolon.
4. Initialise accumulators before the loop: a sum starts at 0, a product at 1, a counter at 0.
5. Guarantee termination: at least one statement inside the loop must move the control variable towards the stopping condition (e.g. `i++`).

Reflex: to trace a loop, build a table with one column per variable and one row per iteration, plus a column for the condition's truth value.

WORKED EXAMPLE 11 **Loops: average of marks with validation**

Write a C program that reads the number *n* of students in a class, then reads *n* marks (each

must be between 0 and 20; force re-entry otherwise) and displays the class average to two decimal places.

SOLUTION

Analysis. The number of marks is known ⇒ outer `for` loop. Each mark must be re-asked until valid, and it is read at least once ⇒ inner `do...while`. The average needs real division, so the sum is divided by *n* as `float`.
Program.

```
#include <stdio.h>
int main(void) {
  int n, i, mark;
  float sum = 0, average;
  printf("Number of students: ");
  scanf("%d", &n);
  for (i = 1; i <= n; i++) {
    do {
      printf("Mark of student %d (0-20): ", i);
      scanf("%d", &mark);
    } while (mark < 0 || mark > 20);
    sum = sum + mark;
  }
  average = sum / n;
  printf("Class average: %.2f\n", average);
  return 0;
}
```

Trace with $n = 3$ and marks 12, 25 (rejected), 14, 7:

i	mark accepted	sum after	remark
1	12	12	–
2	14	26	25 rejected by do…while
3	7	33	–

average = 33/3 = 11.00. Output: `Class average: 11.00`.

Key points. `sum` is declared `float` so that `sum / n` is a real division; with `int sum`, 33/3 would still be exact here but 34/3 would be truncated to 11. The format `%.2f` rounds the display to two decimals. The `do...while` guarantees the mark is requested at least once and repeated while invalid.

METHOD **Handling strings and characters**

1. A C string is a char array ending with the null character `'\0'`. Declare one extra cell: `char name[30];` holds at most 29 usable characters.
2. Read a word with `scanf("%s", name);` (no `&` for an array). Read a full line with `fgets(name, 30, stdin);`.
3. Use `<string.h>` functions: `strlen(s)` (length), `strcpy(dest, src)` (copy), `strcat(dest, src)` (concatenate), `strcmp(a, b)` (compare: returns 0 if equal, < 0 if `a` before `b`).
4. Never compare strings with ==: that compares addresses, not contents. Always use `strcmp`.
5. Single characters: use `<ctype.h>` — `toupper(c)`, `tolower(c)`, `isdigit(c)`, `isalpha(c)`.

WORKED EXAMPLE 12 **Strings: counting vowels in a name**

Write a C program that reads a word (no spaces) and displays the number of vowels it contains, treating upper-case and lower-case vowels alike. Example: `Cameroon` → 4 vowels.

SOLUTION

Analysis. Walk through the string character by character with a `for` loop running from index 0 to `strlen(word) - 1`. Convert each character to lower case once, then test membership in the set $\{a, e, i, o, u\}$.
Program.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(void) {
  char word[50];
```

```
  int i, count = 0;
  char c;
  printf("Enter a word: ");
  scanf("%s", word);
  for (i = 0; i < strlen(word); i++) {
    c = tolower(word[i]);
    if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u') {
      count++;
    }
  }
  printf("Number of vowels: %d\n", count);
  return 0;
}
```

Trace with `word = "Cameroon"` (length 8):

i	word[*i*]	`tolower`	count
0	C	c	0
1	a	a	1
2	m	m	1
3	e	e	2
4	r	r	2
5	o	o	3
6	o	o	4
7	n	n	4

Output: `Number of vowels: 4.`
Key points. Character constants use single quotes (`'a'`), string constants double quotes (`"a"`). `tolower` makes the test case-insensitive without doubling the conditions. Note that `word[i]` accesses character number *i* + 1 because indexing starts at 0.

METHOD **Implementing 1D arrays with loops**

1. Declare with a fixed capacity: `int t[100];` — valid indices are 0 to 99.

2. Fill with a `for` loop from 0 to `n-1`, reading each `t[i]`.
3. Process with a second loop: accumulate, compare, count. To find a maximum, initialise `max = t[0]` then compare with every `t[i]`.
4. Search for a value: loop while index < *n* and value not found; after the loop, the index tells you whether the search succeeded.
5. Never exceed `n-1`: C does not check bounds; `t[n]` reads random memory.

Reflex: one array traversal = one `for` loop; the body contains the per-element treatment.

WORKED EXAMPLE 13 **1D array: highest mark and how many students got it**

A teacher enters the marks of *n* students ($n \leq 50$) into an array. Write a C program that displays the highest mark and the number of students who obtained it.

SOLUTION

Analysis. Two traversals: (1) fill the array; (2) find the maximum; (3) count occurrences of the maximum. Steps (2) and (3) can be merged, but keeping them separate is clearer. Program.

```
#include <stdio.h>
int main(void) {
  float t[50];
  int n, i, count = 0;
  float max;
  printf("Number of students (max 50): ");
  scanf("%d", &n);
  for (i = 0; i < n; i++) {
    printf("Mark %d: ", i+1);
    scanf("%f", &t[i]);
  }
  max = t[0];
  for (i = 1; i < n; i++) {
    if (t[i] > max) max = t[i];
```

```
    }
    for (i = 0; i < n; i++) {
      if (t[i] == max) count++;
    }
    printf("Highest mark: %.2f obtained by %d student(s)\n", max, count);
    return 0;
}
```

Trace with $n = 5$, marks 12.5, 15, 9, 15, 11:

i	$t[i]$	max after pass 2	count after pass 3
0	12.5	12.5	0
1	15	15	1
2	9	15	1
3	15	15	2
4	11	15	2

Output: `Highest mark: 15.00 obtained by 2 student(s).`
Key points. Initialising `max = t[0]` (not 0) is essential: if all marks were negative, `max = 0` would give a wrong answer. The counting loop uses == on floats safely here because the values were read identically; in general, float equality should be tested with a tolerance.

METHOD **Implementing 2D arrays and records**

2D arrays:

1. Declare `float m[10][10];` — first index = row, second = column.
2. Process with nested loops: outer loop over rows `i`, inner loop over columns `j`; element `m[i][j]`.
3. To sum a row, fix `i` and loop over `j`; to sum a column, fix `j` and loop over `i`.

Records (struct):

1. Define the type before `main`: `struct Student { char name[30]; int age; float avg; };`
2. Declare variables: `struct Student s;` or an array `struct Student class[50];` (a simple list of records).
3. Access fields with the dot: `s.avg = 14.5;`, `scanf("%s", s.name);`.

WORKED EXAMPLE 14 **2D array + records: term averages of a class**

A class has n students ($n \le 30$) and each student has marks in 3 subjects. Write a C program that: (a) stores the marks in a 2D array `marks[30][3]`; (b) computes each student's average into a 1D array; (c) stores each student's name and average in an array of records; (d) displays the name and average of the best student.

SOLUTION

Program.

```
#include <stdio.h>
struct Student {
  char name[30];
  float average;
};
int main(void) {
  float marks[30][3], avg[30];
  struct Student class[30];
  int n, i, j, best = 0;
  float sum;
  printf("Number of students: ");
  scanf("%d", &n);
  for (i = 0; i < n; i++) {
    printf("Name of student %d: ", i+1);
    scanf("%s", class[i].name);
    sum = 0;
    for (j = 0; j < 3; j++) {
      printf(" Mark subject %d: ", j+1);
      scanf("%f", &marks[i][j]);
      sum = sum + marks[i][j];
    }
    avg[i] = sum / 3;
```

```
    class[i].average = avg[i];
  }
  for (i = 1; i < n; i++) {
    if (class[i].average > class[best].average) best = i;
  }
  printf("Best student: %s with %.2f\n", class[best].name,
class[best].average);
  return 0;
}
```

Trace with $n = 2$: Amina (12, 14, 16), Bih (10, 11, 9).

Student	marks row			average
Amina	12	14	16	42/3 = 14.00
Bih	10	11	9	30/3 = 10.00

Search for the maximum: `best` starts at 0 (Amina); comparing index 1: 10.00 > 14.00 false, so `best` stays 0. Output: `Best student: Amina with 14.00.`
Key points. Row i of the 2D array holds all the marks of student i: the inner loop over `j` accumulates `sum`, which must be reset to 0 for each student (inside the outer loop, before the inner one) — forgetting this is a classic error that makes averages grow student after student. The array of records `class[]` is a simple list: each element bundles a name and an average, and `best` stores the index of the best record rather than the value, which lets us print both fields.

EXPERIMENT **Guided debugging session**

Scenario: A student writes a program to compute the factorial of a number but gets wrong results for inputs > 12. The code uses `int` for the result.

1. Observe: `int` overflows at 13! (exceeds $2^{31} - 1$). The program compiles but produces negative numbers.
2. Hypothesize: Change `int` to `long long` and use `%lld` in `printf`/`scanf`.
3. Test: Run with $n = 13$, then $n = 20$. Verify against known values (13! = 6227020800).
4. Conclude: Always match the data type to the expected range. For arbitrarily large integers, a big-integer library is needed.

Lesson: Debugging is not only fixing syntax errors; it is validating logic and data representation against the problem's constraints.

METHOD **Implementing subroutines (functions) and debugging**

1. Identify repeated or coherent treatments and isolate them as functions: a function has a name, parameters (inputs) and a return type (output).
2. Declare the prototype above `main` (e.g. `float average(float a, float b, float c);`) and define the function after `main`.
3. Call it with actual arguments: `m = average(x, y, z);`.
4. Parameters are passed by value: the function works on copies. To modify a caller's variable, pass its address (pointer) or use an array (arrays are always passed by reference).
5. Debugging method: read compiler messages from the first one; add temporary `printf` to inspect variable values; test with simple data whose result you know; check boundary cases (first/last index, $n = 1$, empty input).

Reflexes: a function that returns nothing has type `void`; every non-`void` function must end with `return value;`.

WORKED EXAMPLE 15 **Subroutines: prime numbers up to N — integration activity**

Write a complete C program built around subroutines that: (a) reads a positive integer N with validation; (b) uses a function `int isPrime(int k)` returning 1 if k is prime and 0 otherwise; (c) uses a `void` function `printPrimes(int n)` that displays all prime numbers from 2 to n and their count. Test with $N = 20$.

SOLUTION

Program.

```
#include <stdio.h>
int isPrime(int k);
void printPrimes(int n);
int main(void) {
  int N;
  do {
    printf("Enter a positive integer N: ");
    scanf("%d", &N);
  } while (N <= 0);
```

```
  printPrimes(N);
  return 0;
}
int isPrime(int k) {
  int d;
  if (k < 2) return 0;
  for (d = 2; d * d <= k; d++) {
    if (k % d == 0) return 0;
  }
  return 1;
}
void printPrimes(int n) {
  int k, count = 0;
  for (k = 2; k <= n; k++) {
    if (isPrime(k)) {
      printf("%d ", k);
      count++;
    }
  }
  printf("\nTotal: %d prime(s)\n", count);
}
```

Trace for $N = 20$. `isPrime` tests each k with divisors d up to $\sqrt{k}$:

k	divisors tried ($d * d \le k$)	result
2	none ($4 > 2$)	prime
4	$d = 2$: $4\%2 = 0$	not prime
9	$d = 2$: no; $d = 3$: $9\%3 = 0$	not prime
13	$d = 2, 3$: no divisor	prime

Output: `2 3 5 7 11 13 17 19` then `Total: 8 prime(s).`
Key points and debugging discussion.

- Efficiency: testing divisors only up to $\sqrt{k}$ (written `d * d <= k` to avoid `math.h`) reduces the work dramatically for large k.
- Early exit: `return 0` inside the loop stops the function as soon as a divisor is found — no need to test further.
- Common bug 1: forgetting `if (k < 2) return 0;` would wrongly declare 0 and 1 prime.
- Common bug 2: writing the loop as `d <= k/2` works but is slower; writing `d < sqrt(k)` misses the case $k = d^2$ (e.g. 9) if not careful — `d * d <= k` is the safe form.
- Modularity: `main` only orchestrates; each function does one job. This is exactly the structure expected in the GCE integration tasks: input validation, processing subroutines, clear output.

WARNING **Final checklist before running any C program at the GCE**

- Every variable declared before use; every statement ends with `;`.
- `&` present in `scanf` for `int`/`float`/`char` (but not for arrays/strings).
- Comparison written `==`, assignment `=`.
- Array indices between 0 and size-1; loops terminate.
- Strings compared with `strcmp`, copied with `strcpy`.
- Braces balanced; prototypes declared; non-`void` functions return a value.
- Program tested with at least one normal case and one boundary case.

9 EXERCISES

9.1 I check my knowledge

EXERCISE 8 **MCQ: C fundamentals**

For each question, choose the single correct answer and justify your choice in one sentence.

1. Which of the following is the correct way to declare an integer variable in C?

   (a) `integer x;` (b) `int x;` (c) `x : int;` (d) `declare x;`

2. The function used to display output on the screen in C is:

   (a) `scanf` (b) `read` (c) `printf` (d) `display`

3. Which header file must be included to use `printf` and `scanf`?

   (a) `math.h` (b) `string.h` (c) `stdlib.h` (d) `stdio.h`

4. The expression `7 % 3` in C evaluates to:

   (a) 2 (b) 1 (c) 2.33 (d) 0

5. In C, the index of the first element of an array declared as `int t[10];` is:

   (a) 1 (b) 0 (c) 10 (d) -1

EXERCISE 9 **True or false**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. A C program must always contain a function called `main`.
2. The instruction `if (x = 10)` compares the variable `x` with the value 10.
3. A `do ... while` loop executes its body at least once, even if the condition is false from the start.
4. A string in C is a built-in data type exactly like `int` or `float`.
5. The elements of a `struct` may have different data types.

EXERCISE 10 **Definitions and vocabulary**

Define each of the following terms in your own words, giving a short C example for each:

1. **Compiler**
2. **Syntax error** and **logic error** (state clearly the difference)
3. **Sentinel value**
4. **Function prototype**
5. **Array** and **record** (`struct`)

EXERCISE 11 **Direct evaluation of expressions**

Given the declarations `int a = 7, b = 2;` and `float x = 7.0;`, evaluate each of the following C expressions, giving the value and its type:

1. `a / b`
2. `x / b`
3. `a % b`
4. `a > b && b > 0`
5. `(float)a / b`
6. `a = a + 3` (state the new value of `a`)

9.2 I practise

EXERCISE 12 **The market stall bill**

A trader at Mokolo market sells items by quantity. Write a complete C program that:

1. reads the unit price (in FCFA) and the quantity of an item bought;
2. computes the total amount;
3. applies a discount of 5% if the total exceeds 10 000 FCFA;
4. displays the discount granted and the final amount to pay.

Test your program with a unit price of 1 500 FCFA and a quantity of 8.

EXERCISE 13 Sentinel loop with accumulators

Write a C program that reads integers entered by the user until the value -1 is typed (the sentinel -1 must not be counted). The program then displays:

1. the number of values entered;
2. the sum of the values;
3. the maximum value entered;
4. the average of the values (as a real number).

Take care of the special case where the user types -1 immediately.

EXERCISE 14 From `while` to `for`

The following fragment prints the multiplication table of 7:

```
int i = 1;
while (i <= 12) {
  printf("%d x 7 = %d\n", i, i * 7);
  i++;
}
```

1. Rewrite this fragment using a `for` loop.
2. Trace the first three iterations in a table with columns: value of `i` (test), condition true/false, output produced, value of `i` after increment.

EXERCISE 15 Working on a name

Write a C program that reads a student's surname (a single word) and then:

1. counts and displays the number of vowels it contains;
2. converts the name to uppercase and displays it;
3. checks whether the name is a palindrome (reads the same in both directions, e.g. `ANA`) and displays an appropriate message.

You may use the functions of `string.h` and `ctype.h`.

9.3 I prepare for the GCE

EXERCISE 16 **Class marks processing — 1D arrays (8 marks)**

A teacher stores the marks of 20 students in an array `float marks[20];`.

1. Write a C code fragment that fills the array with marks entered by the user, rejecting any mark outside the range [0, 20]. (2 marks)
2. Write a fragment that displays all the marks and computes the class average. (2 marks)
3. Write a fragment that counts how many marks are greater than or equal to 10. (2 marks)
4. Write a fragment that finds and displays the highest mark together with its position in the array (the position of the first student being 1). (2 marks)

EXERCISE 17 **Debugging a faulty program (10 marks)**

The following C program was written by a candidate. It contains exactly five errors: a missing semicolon, == written as =, an array index out of bounds, a missing `&` in a `scanf` call, and a missing `break` in a `switch`.

```
#include <stdio.h>
int main() {
  int t[5]; int n, i, choice;
  scanf("%d", n)
  for (i = 1; i <= 5; i++) t[i] = i * n;
  if (n = 0) printf("zero\n");
  scanf("%d", &choice);
  switch (choice) {
    case 1: printf("one\n");
    case 2: printf("two\n"); break;
    default: printf("other\n");
  }
  return 0;
}
```

1. Identify the five errors, giving for each one the line concerned and the type of error (syntax, logic or runtime). (5 marks)
2. Explain what actually happens at run time because of the error `if (n = 0)`. (2 marks)
3. Write out the fully corrected program. (3 marks)

EXERCISE 18 **Student records and subroutines (12 marks)**

A school wishes to manage the results of a class of 35 students. Each student is described by a name (string), an age (integer) and an average mark (real).

1. Define in C a structure `Student` containing the three fields above. (2 marks)
2. Declare an array `classList` able to hold the 35 records, and write a code fragment that fills the array with data entered by the user. (3 marks)
3. Write a function `float average3(float a, float b, float c)` that returns the average of three marks. (2 marks)
4. Write a subroutine (function) that receives a `Student` and displays `ADMITTED` if the student's average is at least 10, and `REPEATED` otherwise. State whether you pass the structure by value or by pointer, and justify. (3 marks)
5. Write the `main` program that uses the array and the subroutines to display the names of all admitted students. (2 marks)

10 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: C fundamentals**

1. Correct answer: (b) `int x;` — In C, a variable is declared by writing its type followed by its identifier and a semicolon; `integer` is the Pascal keyword, `x : int;` is Pascal syntax and `declare` does not exist in C.
2. Correct answer: (c) `printf` — `printf` is the standard formatted output function of the C library; `scanf` is for input, and `read` / `display` are not standard C output functions.
3. Correct answer: (d) `stdio.h` — `stdio.h` (standard input/output) declares `printf` and `scanf`; `math.h` is for mathematics, `string.h` for strings and `stdlib.h` for general utilities.
4. Correct answer: (b) 1 — `%` is the remainder of integer division: $7 = 3 \times 2 + 1$, so `7 % 3` is 1.
5. Correct answer: (b) 0 — In C, array indices always start at 0, so the elements of `int t[10];` are `t[0]` to `t[9]`.

SOLUTION **Exercise 2 — True or false**

1. True. Every executable C program must contain exactly one `main` function: it is the entry point where execution begins.
2. False. `if (x = 10)` is an assignment: it stores 10 in `x` and the condition is then the value 10, which is non-zero, hence always true. Comparison requires the double equals `x == 10`.
3. True. In a `do ... while` loop the body is executed first and the condition is tested only afterwards, so the body always runs at least once.
4. False. C has no built-in string type; a string is an array of `char` terminated by the null character `'\0'`, manipulated with functions from `string.h`.
5. True. Unlike an array, whose elements all share one type, the fields of a `struct` may be of different types (e.g. a `char` array, an `int` and a `float`).

SOLUTION **Exercise 3 — Definitions and vocabulary**

1. Compiler: a program that translates the whole C source code into machine language (an executable file) before execution, reporting all syntax errors found. Example: `gcc`

`hello.c -o hello` compiles `hello.c` into the executable `hello`.

2. Syntax error / logic error: a syntax error is a violation of the grammar of C (missing semicolon, undeclared variable) detected by the compiler, so the program does not compile; a logic error is a mistake in the reasoning: the program compiles and runs but produces wrong results. Example of syntax error: `int x = 5` (no semicolon). Example of logic error: writing `if (x = 0)` instead of `if (x == 0)`.
3. Sentinel value: a special value, impossible as normal data, that signals the end of input in a loop. Example: reading marks until `-1` is typed: `while (mark != -1)`.
4. Function prototype: the declaration of a function (return type, name, parameter types) placed before `main` so the compiler can check calls; the body is defined later. Example: `float average3(float a, float b, float c);`
5. Array / record: an array is a collection of elements of the same type accessed by an index, e.g. `int t[10];`. A record (`struct`) groups fields of possibly different types under one name, e.g. `struct Student { char name[30]; int age; float avg; };`

SOLUTION **Exercise 4 — Direct evaluation of expressions**

Recall: `a = 7` (`int`), `b = 2` (`int`), `x = 7.0` (`float`).

1. `a / b`: integer division 7/2 ⇒ value 3, type `int` (the fractional part is discarded).
2. `x / b`: `b` is converted to `float` (mixed arithmetic) ⇒ value 3.5, type `float`.
3. `a % b`: remainder of 7 ÷ 2 ⇒ value 1, type `int`.
4. `a > b && b > 0`: 7 > 2 is true and 2 > 0 is true ⇒ value 1 (true), type `int` (C has no boolean type; conditions yield 0 or 1).
5. `(float)a / b`: the cast converts `a` to 7.0 before the division ⇒ value 3.5, type `float`.
6. `a = a + 3`: assignment, not comparison; the expression has value 10 (type `int`) and the new value of `a` is 10.

SOLUTION **Exercise 5 — The market stall bill**

```
#include <stdio.h>
int main() {
  float unitPrice, total, discount = 0.0, finalAmount;
  int quantity;
```

```
  printf("Unit price (FCFA): ");
  scanf("%f", &unitPrice);
  printf("Quantity: ");
  scanf("%d", &quantity);
  total = unitPrice * quantity;
  if (total > 10000) {
    discount = 0.05 * total;
  }
  finalAmount = total - discount;
  printf("Total: %.0f FCFA\n", total);
  printf("Discount: %.0f FCFA\n", discount);
  printf("Amount to pay: %.0f FCFA\n", finalAmount);
  return 0;
}
```

Test with 1 500 FCFA and quantity 8: total = 1500 × 8 = 12 000 FCFA. Since 12 000 > 10 000, discount = 0.05×12 000 = 600 FCFA, and the final amount is 12 000−600 = 11 400 FCFA. Output: `Total: 12000 FCFA`, `Discount: 600 FCFA`, `Amount to pay: 11400 FCFA`.

SOLUTION **Exercise 6 — Sentinel loop with accumulators**

```
#include <stdio.h>
int main() {
  int value, count = 0, sum = 0, max;
  float average;
  printf("Enter integers (-1 to stop): ");
  scanf("%d", &value);
  while (value != -1) {
    if (count == 0 || value > max) max = value;
    sum = sum + value;
```

```
    count++;
    scanf("%d", &value);
  }
  if (count == 0) {
    printf("No value was entered.\n");
  } else {
    average = (float)sum / count;
    printf("Count: %d\n", count);
    printf("Sum: %d\n", sum);
    printf("Maximum: %d\n", max);
    printf("Average: %.2f\n", average);
  }
  return 0;
}
```

Key points: the first `scanf` is placed before the loop (priming read) so that an immediate -1 is handled; the maximum is initialised with the first real value (`count == 0` test), never with 0, which would fail if all values were negative; the cast `(float)sum` forces real division; the case `count == 0` avoids a division by zero.

SOLUTION **Exercise 7 — From `while` to `for`**

1. Equivalent `for` loop:

```
for (int i = 1; i <= 12; i++) {
  printf("%d x 7 = %d\n", i, i * 7);
}
```

The three parts of the `for` header correspond exactly to the initialisation `i = 1`, the continuation condition `i <= 12` and the increment `i++` of the `while` version.

2. Trace of the first three iterations:

`i` (test)	Condition `i <= 12`	Output produced	`i` after increment
1	true	`1 x 7 = 7`	2
2	true	`2 x 7 = 14`	3
3	true	`3 x 7 = 21`	4

The loop stops when `i` becomes 13, since 13 ≤ 12 is false.

SOLUTION **Exercise 8 — Working on a name**

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
  char name[50], upper[50];
  int i, vowels = 0, palindrome = 1, len;
  printf("Enter the surname: ");
  scanf("%s", name);
  len = strlen(name);
  for (i = 0; i < len; i++) {
    char c = tolower(name[i]);
    if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u'||c=='y') vowels++;
    upper[i] = toupper(name[i]);
  }
  upper[len] = '\0';
  printf("Number of vowels: %d\n", vowels);
  printf("Uppercase: %s\n", upper);
  for (i = 0; i < len / 2; i++) {
    if (tolower(name[i]) != tolower(name[len-1-i])) palindrome = 0;
  }
  if (palindrome) printf("%s is a palindrome.\n", name);
```

```
  else printf("%s is not a palindrome.\n", name);
  return 0;
}
```

Remarks: `strlen` gives the length without the `'\0'`; the uppercase copy must be explicitly terminated with `'\0'`; comparing `tolower` of both ends makes the palindrome test case-insensitive, so `Ana` is also recognised. Example run: input `Atangana` gives 4 vowels, `ATANGANA`, "not a palindrome".

SOLUTION **Exercise 9 — Class marks processing (8 marks)**

1. Filling with validation (2 marks):

```
int i;
for (i = 0; i < 20; i++) {
  do {
    printf("Mark of student %d (0-20): ", i + 1);
    scanf("%f", &marks[i]);
  } while (marks[i] < 0 || marks[i] > 20);
}
```

Examiner expectations (1 + 1): correct loop over the 20 cells; a `do ... while` (or equivalent) that re-asks while the mark is outside [0, 20].

2. Display and average (2 marks):

```
float sum = 0;
for (i = 0; i < 20; i++) {
  printf("Student %d: %.2f\n", i + 1, marks[i]);
  sum = sum + marks[i];
}
printf("Class average: %.2f\n", sum / 20);
```

Expectations: accumulator initialised to 0; division by 20 producing a `float` (here automatic since `sum` is `float`).

3. Counting marks ≥ 10 (2 marks):

```
int passed = 0;
for (i = 0; i < 20; i++) {
```

```
    if (marks[i] >= 10) passed++;
}
printf("Marks >= 10: %d\n", passed);
```

4. Highest mark and its position (2 marks):

```
int pos = 0;
for (i = 1; i < 20; i++) {
    if (marks[i] > marks[pos]) pos = i;
}
printf("Highest mark %.2f at position %d\n", marks[pos], pos + 1);
```

Expectations: the maximum is initialised with the first element (index 0), not with 0; the displayed position is `pos + 1` because students are numbered from 1. Using `>` (strict) keeps the first occurrence in case of a tie.

SOLUTION **Exercise 10 — Debugging a faulty program (10 marks)**

1. The five errors (5 marks, 1 per error):

Line	Error	Type
`scanf("%d", n)` (no ;)	Missing semicolon	Syntax
`for (i = 1; i <= 5; i++) t[i] = ...`	Writes `t[5]`, out of bounds for `int t[5]` (valid indices 0–4)	Runtime (undefined behaviour)
`if (n = 0)`	Assignment instead of comparison ==	Logic
`scanf("%d", n)`	Missing `&` before `n`	Runtime (crash / undefined behaviour)
`case 1: printf("one\n");`	Missing `break`: falls through into `case 2`	Logic

2. Effect of `if (n = 0)` (2 marks): the expression `n = 0` is an assignment: it overwrites `n` with 0, destroying the value read from the user. The value of an assignment expression is the assigned value, here 0, which C interprets as false; therefore the message `zero` is never displayed, whatever the user typed. The test is useless and the data are corrupted.

3. Corrected program (3 marks):

```
#include <stdio.h>
int main() {
```

```
  int t[5]; int n, i, choice;
  scanf("%d", &n);
  for (i = 0; i < 5; i++) t[i] = i * n;
  if (n == 0) printf("zero\n");
  scanf("%d", &choice);
  switch (choice) {
    case 1: printf("one\n"); break;
    case 2: printf("two\n"); break;
    default: printf("other\n");
  }
  return 0;
}
```

Examiner expectations: the corrected loop must run from 0 to 4 (`i < 5`); both `scanf` calls use `&`; `==` in the test; `break` after `case 1`.

SOLUTION **Exercise 11 — Student records and subroutines (12 marks)**

1. Structure definition (2 marks):

```
struct Student {
  char name[30];
  int age;
  float average;
};
```

Expectations: keyword `struct`, three fields of suitable types, terminating semicolon after the closing brace.

2. Array declaration and filling (3 marks):

```
struct Student classList[35];
int i;
for (i = 0; i < 35; i++) {
  printf("Name of student %d: ", i + 1);
  scanf("%s", classList[i].name);
```

```
  printf("Age: ");
  scanf("%d", &classList[i].age);
  printf("Average: ");
  scanf("%f", &classList[i].average);
}
```

Expectations: access to fields with the dot operator; note that `name` needs no `&` (an array name is already an address) whereas `age` and `average` do.

3. Function `average3` (2 marks):

```
float average3(float a, float b, float c) {
  return (a + b + c) / 3;
}
```

4. Decision subroutine (3 marks):

```
void verdict(struct Student s) {
  if (s.average >= 10) printf("%s: ADMITTED\n", s.name);
  else printf("%s: REPEATED\n", s.name);
}
```

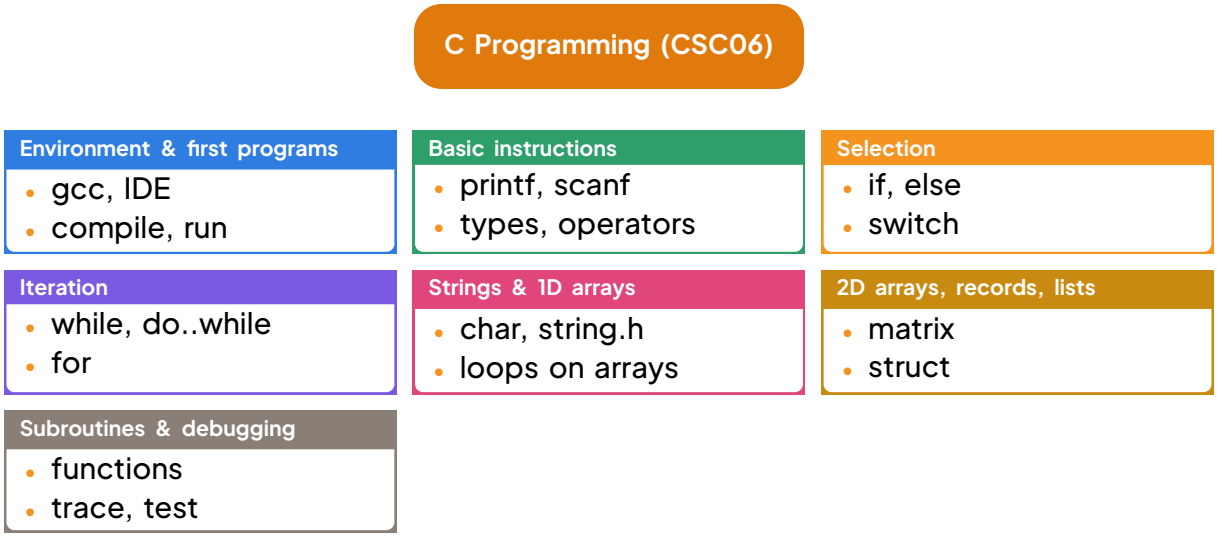
Passing mode: here the structure is passed by value: the function only reads the fields and does not modify the original record, so a copy is safe and simple. (Passing a pointer, e.g. `void verdict(struct Student *s)` with `s->average`, would be preferred for very large structures or if the function had to modify the record, since it avoids copying.)

5. Main program (2 marks):

```
int main() {
  struct Student classList[35];
  int i;
  /* filling loop of question 2 here */
  printf("Admitted students:\n");
  for (i = 0; i < 35; i++) {
    if (classList[i].average >= 10)
      printf("%s\n", classList[i].name);
  }
  return 0;
}
```

Expectations: reuse of the array and of the decision criterion; a variant calling `verdict(classList[i])` for every student is also accepted. Mark scheme summary: structure 2; array + filling 3; `average3` 2; verdict + justification 3; main 2; total 12.

11 SUMMARY SHEET



Mind map of chapter CSC06: writing, debugging and running C programs.

FICHE BILAN **L'essentiel à connaître pour l'examen**

1. Key definitions

- **Program**: a sequence of instructions written in a programming language to solve a problem.
- **Compiler** (e.g. `gcc`): translates the whole C source file (`.c`) into an executable file; a **syntax error** is detected at compile time, a **logic error** only at run time.
- **Variable**: named memory location with a type (`int`, `float`, `double`, `char`), a name and a value.
- **Array**: fixed-size collection of elements of the same type, indexed from 0 to $n-1$.
- **String**: array of `char` ending with the null character `'\0'`.
- **Record** (`struct`): groups fields of different types under one name.
- **Function** (subroutine): named, reusable block; procedure returns `void`, function returns a value with `return`.
- **Debugging**: locating and correcting errors using trace tables, test data and `printf` checks.

2. Essential syntax to know by heart

- Skeleton: `#include <stdio.h>` then `int main(){ ... return 0; }`.

- Output: `printf("Sum = %d\n", s);` — formats: `%d` (int), `%f` / `%.2f` (float), `%c` (char), `%s` (string).
- Input: `scanf("%d", &x);` — never forget the `&` (address of) for `scanf`.
- Selection: `if (cond) {...} else if (cond) {...} else {...};` `switch(n){ case 1: ...; break; default: ...}`.
- Loops: `while (cond)`, `do {...} while (cond);` (body runs at least once), `for (i=0; i<n; i++)`.
- Conditions: `==`, `!=`, `<`, `<=`, `>`, `>=`; combined with `&&` (AND), `||` (OR), `!` (NOT).
- Arrays: declaration `int t[10];`, `float m[3][4];`; traversal always with a `for` loop (two nested loops for 2D).
- Strings: `char name[30];` with `strlen`, `strcpy`, `strcat`, `strcmp` from `<string.h>`; single character test with `<ctype.h>` (`isdigit`, `toupper`).
- Records: `struct Student{ char name[30]; float avg; };` then access with the dot: `s.avg`.
- Functions: prototype before `main`, definition after; parameters passed by value (a copy) unless an array or address is passed.

3. Methods

- Problem to program: analyse (inputs, processing, outputs) → write the algorithm in pseudocode → translate to C → compile → test with well-chosen data → debug.
- Trace table: one column per variable, one row per loop iteration; the indispensable tool to verify loops and find logic errors.
- Classic patterns: sum/average of an array (accumulator initialised to 0), maximum (initialise with first element), search (loop + flag), counting with a condition.
- Testing: use normal values, boundary values (0, n - 1) and invalid values.

4. Common mistakes (GCE traps!)

- Writing `if (x = 5)` (assignment) instead of `if (x == 5)` (comparison).
- Forgetting `&` in `scanf`, or the semicolon at the end of a statement, or `break` in a `switch` case.
- Accessing `t[n]` in an array of size n: valid indices stop at n - 1 (out-of-range access = undefined behaviour).
- Integer division: `5/2` gives `2`, not `2.5`; cast with `(float)a/b`.
- Comparing strings with `==` instead of `strcmp(a,b)==0`.
- Loop counter not updated ⇒ infinite loop; wrong initialisation of an accumulator.

- Forgetting `'\0'` space when sizing a string array.

5. GCE tips
   - Read the question twice: identify required inputs, outputs and constraints before coding.
   - Always declare and initialise variables; give meaningful names (`total`, `count`).
   - In paper 2 (practical/algorithm questions), a clean trace table often earns method marks even if the final answer is wrong.
   - Comment your code (`//`) and indent properly: clarity is rewarded.
   - Check boundary cases in loops: `i < n` versus `i <= n` is a classic exam distinction.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS
OnBuch+